

HyperCLIP++: Fine-tuning CLIP for Open-vocabulary Semantic Segmentation in Hyperbolic Space

Zelin Peng, Zhengqin Xu, Changsong Wen, Yu Huang, Yaoming Wang, Xiaokang Yang, *Fellow, IEEE*, Wei Shen

Abstract—CLIP, a foundational vision-language model, has emerged as a powerful tool for open-vocabulary semantic segmentation. While freezing CLIP’s text encoder is known to preserve its generalization capability, recent studies show that fine-tuning both CLIP’s text and image encoders jointly significantly enhances segmentation performance, especially for classes from open sets. In this work, we explain this phenomenon from the perspective of hierarchy alignment, since during fine-tuning, the hierarchical level of image embeddings shifts from image-level to pixel-level. We achieve this by leveraging hyperbolic space, which naturally encodes hierarchical structures. Our key observation is that, during fine-tuning, the hyperbolic radius of CLIP’s text embeddings decreases, facilitating better alignment with the pixel-level granularity of visual data. Building on this, we propose HyperCLIP++, a novel and parameter-efficient adaptation strategy. HyperCLIP++ directly adjusts the hyperbolic radius of CLIP’s embeddings via scaling transformations to achieve a hierarchy alignment to the target task, i.e., segmentation. To ensure this hierarchy alignment is effected consistently across both modalities and preserves their cross-modal alignment during training, HyperCLIP++ integrates a Dual Cross-Relation Communication (DCRC) module that synchronizes these adjustments between the vision and text pathways. Our experiments show that HyperCLIP++ achieves state-of-the-art performance across three benchmarks while fine-tuning only approximately 5% of CLIP’s total parameters. More importantly, we observe that after adjustment, CLIP’s text embeddings exhibit a relatively fixed hyperbolic radius across datasets, suggesting that the hierarchical level required for this segmentation task might be quantified using the hyperbolic radius. The code is available at <https://github.com/SJTU-DeepVisionLab/HyperCLIP-Plus-Plus>.

Index Terms—Open-vocabulary semantic segmentation, vision-language model, hyperbolic space, dual cross relation communication.

1 INTRODUCTION

The goal of open-vocabulary semantic segmentation is to develop a segmentation model capable of labeling each pixel in an image with categories that extend beyond a predefined closed set, based on textual descriptions. Vision-language foundation models [1]–[20], particularly CLIP [4], are frequently employed to provide open-vocabulary recognition capabilities. Consequently, open-vocabulary semantic segmentation fundamentally involves transferring these vision-language foundation models, originally trained with image-level supervision, to achieve pixel-level predictions.

Current methods [21]–[24] typically fine-tune CLIP on a closed set with segmentation annotations, i.e., COCO [25], to equip it with the segmentation capability. A prevailing opinion is that simply freezing CLIP’s text encoder maximally preserves the powerful text embeddings of a vast array of classes, which is believed to be beneficial for good generalization on open sets. Many studies follow

this opinion [24], [26], [27]. However, recent studies [22], [23], [28] offer an alternative perspective on fine-tuning: simultaneously fine-tuning both the image and text encoders results in superior segmentation performance on open sets compared to former approaches. To explain and resolve this seemingly paradoxical phenomenon, we first leverage a non-Euclidean manifold—specifically, hyperbolic space [29]—to understand why fine-tuning the text encoder appears to improve segmentation performance on open sets. Building on this understanding, our objective is to design an explainable fine-tuning strategy with a minimal number of introduced tunable parameters, which is able to equip CLIP with strong segmentation performance while minimizing the loss of its generalization ability.

Existing research [30] has found that images inherently exhibit hierarchical structures, consisting of multiple levels: pixels, patches, objects, entire scenes, etc. Given that, in open-vocabulary semantic segmentation, CLIP’s image embeddings transition from image level to pixel level, we infer that the performance improvement brought by the recent studies [22], [23] could be attributed to adjusting CLIP’s text embeddings from their original hierarchical level to better align with the level represented by pixels, thereby enhancing cross-modal alignment. However, most existing fine-tuning methods operate in Euclidean space, which makes it challenging to quantify such a transition through different hierarchical levels. Hyperbolic spaces have gained significant interest in recent years, owing to their ability to

This work was supported by the NSFC under Grant 62322604 and 62576207. Z. Peng, C. Wen, Y. Huang, X. Yang and W. Shen are with the MoE Key Lab of Artificial Intelligence, AI Institute, School of Computer Science, Shanghai Jiao Tong University, Shanghai 200240, China (E-mail: {zelin.peng, changsong, yellowfish, xkyang, wei.shen}@sjtu.edu.cn).

Z. Xu is with the State Key Laboratory of Infrared Physics, Shanghai Institute of Technical Physics, Chinese Academy of Science, Shanghai, 200083, China (E-mail: fate311@sjtu.edu.cn).

Y. Wang is with Meituan, Shanghai, 200051, China (E-mail: wangyaoming03@meituan.com).

Manuscript received April 19, 2021; revised August 16, 2021.

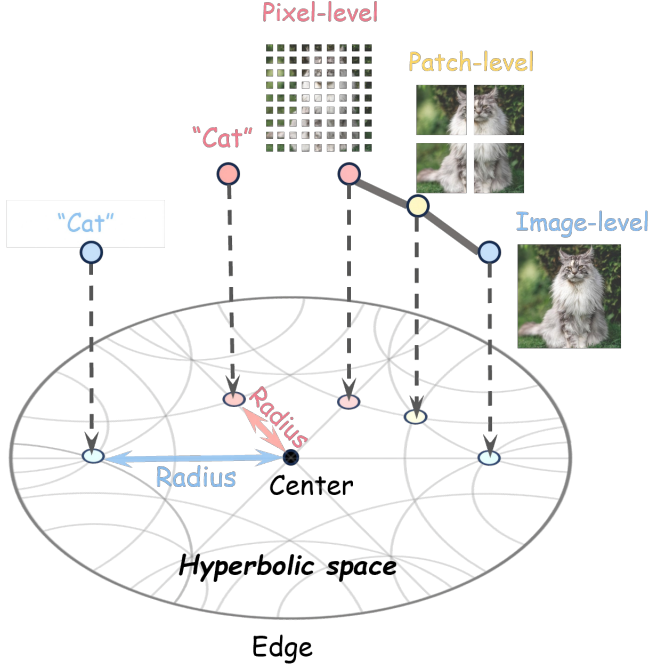


Fig. 1: **Visualization of hyperbolic space by a Poincaré ball model:** The height of points reflects the hierarchical level of input data. The black segments refer to the shortest path between the points in the Poincaré ball model. The red arrow line denotes the hyperbolic radius of text embeddings. Embeddings closer to the center have a smaller radius, representing low-level visual information (e.g., pixels), while points farther from the center have a larger radius, representing high-level visual information (e.g., images). During fine-tuning, our hyperbolic scaling transformations simultaneously shift both visual and text embeddings toward smaller radii (pixel-level), while the DCRC module synchronizes these adjustments across modalities to preserve cross-modal alignment (at the same height). (Best viewed in color).

naturally and compactly encode hierarchical structures [31]–[33], as shown in Fig. 1. The hyperbolic radius, defined as the distance from a point to the center in hyperbolic space, represents its hierarchical level [34]. Points closer to the center (smaller radius) correspond to low-level visual embeddings, e.g., those from pixels, while points near the edge (larger radius) correspond to high-level visual embeddings, e.g., those from images.

Motivated by this, we project the text embeddings into hyperbolic space and visualize the changes in their radii before and after fine-tuning, as shown in Fig. 2. This fine-tuning is realized using the state-of-the-art method provided in [23]. Specifically, one can observe that the radius after fine-tuning is smaller than that of pre-trained CLIP. In particular, we identify a pattern during fine-tuning: the hierarchical level of the text embeddings adjusts in conjunction with the hierarchical level of the embeddings provided by the image encoder. Our finding brings a novel perspective for fine-tuning CLIP: Adjusting the hyperbolic radius of text embeddings from edge to center endowed CLIP with segmentation ability.

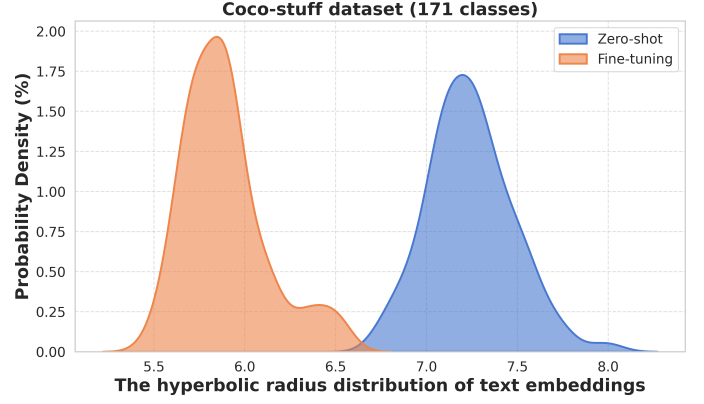


Fig. 2: **Understanding fine-tuning CLIP's text encoder from the perspective of hyperbolic radius.** We use hyperbolic radius to illustrate the change in the hierarchical level of 171 classes after fine-tuning. We observe that the hyperbolic radius of each class becomes smaller, suggesting that adjusting the hyperbolic radius equips CLIP with segmentation ability (Best viewed in color).

Building on this analysis, we propose a new fine-tuning strategy for CLIP in hyperbolic space, termed HyperCLIP++. Our method directly adjusts the hyperbolic radii of CLIP embeddings to align toward the granularity level required for segmentation. It is designed with two key components: (1) *Hierarchy alignment within each modality.* To match the pixel-level nature of the task, we apply this hierarchy alignment to both modalities, as segmentation requires the visual embedding to also shift from image-level to pixel-level. To this end, HyperCLIP++ integrates a set of lightweight, block-diagonal scaling matrices into both of the encoder's vision and text pathways. Through Möbius matrix multiplication, these matrices apply targeted scaling transformations on embeddings, explicitly controlling their hyperbolic radii and ensuring alignment with the task's hierarchical level; (2) *Hierarchy alignment across modalities.* While the scaling matrices regulate intra-modality hierarchy alignment, we further introduce a Dual Cross-Relation Communication (DCRC) module to couple the adjustments of visual and textual embeddings. By enabling structured interactions between the learnable matrices of both encoders, DCRC synchronizes text–vision interaction: as visual embeddings become more fine-grained, text embeddings adapt accordingly, and vice versa. This enforces consistent cross-modal hierarchy alignment, leading to more precise modality coherence within the fine-tuned CLIP.

Extensive experiments demonstrate that our method sets new state-of-the-art results in open-vocabulary semantic segmentation across three benchmarks, while fine-tuning only approximately 5% of the total parameters of CLIP. More importantly, we observe that after adjustment, CLIP's text embeddings maintain a relatively fixed hyperbolic radius across different datasets. This suggests that the hierarchical level of the segmentation task might be quantified using the hyperbolic radius, warranting further exploration.

Relation to the preliminary conference version. This manuscript is an extended version of our previous conference paper, i.e., HyperCLIP [35]. The main contributions

beyond the original version are summarized as follows. First, we observe that the original work primarily ensures hierarchy alignment between the model (i.e., CLIP) and the target task (i.e., segmentation), but lacked precise control over cross-modal alignment. Here, we introduce the DCRC module, which explicitly enhances cross-modal communication among all learnable scaling matrices, thereby improving cross-modal alignment with negligible computational overhead. Second, we extend our investigation to encompass CLIP’s visual embeddings, projecting them into the same hyperbolic space as their text counterparts. This joint visualization presented in Fig. 6 reveals a crucial insight: The proximity of their respective hyperbolic radii serves as a quantitative proxy of the cross-modal alignment, offering a new geometric perspective on how modalities interact during fine-tuning. Finally, we demonstrate the effectiveness and superiority of the proposed DCRC module and HyperCLIP++ through extensive experiments.

2 RELATED WORK

2.1 Open-vocabulary Semantic Segmentation

Prior works on open-vocabulary semantic segmentation typically tackle this task by leveraging CLIP [4]. Early efforts, e.g., [36], directly fine-tune CLIP on mainstream segmentation datasets, like COCO [25]. However, these approaches argue that fully fine-tuning CLIP’s encoder significantly diminishes its ability to generalize to unseen classes. To mitigate this issue, some methods [37]–[40] take the opposite approach, fine-tuning an additional mask generator [41] for segmentation while keeping CLIP frozen to preserve its generalization capabilities. However, this approach, with its frozen parameter space, lacks segmentation awareness, leading to a misalignment between regions and text descriptions [27]. Other studies [21], [23], [24] propose more sophisticated solutions that fine-tune only specific parameters, e.g., certain layers of CLIP, enabling pixel-level predictions while keeping the majority of CLIP’s parameters fixed to minimize the loss of generalization. While these approaches show significant advantages, they often rely on very small learning rates, implicitly promoting minimal deviations from the pre-trained CLIP, which limits segmentation performance. Meanwhile, recent works explore alternative strategies to improve vision-text alignment for segmentation: [42] proposes a collaborative optimization framework that jointly refines vision-text representations, and [43] adaptively adjusts inter-class affinity between image and text feature spaces. In summary, there is still limited understanding of how fine-tuning influences CLIP’s representations and why fine-tuning CLIP’s text encoder appears to enhance segmentation performance. In this paper, we aim to provide several new insights that reveal the underlying logic by incorporating a novel perspective: hyperbolic space.

2.2 Learning in Hyperbolic Spaces

Unlike Euclidean geometry, hyperbolic space can be viewed as the continuous analog of a tree [44], enabling a hierarchical understanding of knowledge structures. Previous research has demonstrated the advantages of representing such hierarchical relationships in various domains, including molecular structures [45], 3D data [46], [47], text

data [48], [49], and images [50], [51]. Furthermore, several studies have investigated hyperbolic learning in vision tasks to model hierarchical relationships across different modalities [33], [52], [53], showcasing the benefits of capturing the semantic hierarchy among various data types. Nevertheless, while most studies, e.g., HyperLoRA [54], leverage hyperbolic space as a static framework for better representing hierarchical data, this paper seeks to capture changes in the granularity of CLIP’s representations through hyperbolic radius adjustment and maintaining cross-modal alignment. This approach potentially complements current fine-tuning efforts in open-vocabulary tasks by offering a novel hyperbolic perspective.

3 PRELIMINARY BACKGROUND

3.1 Hyperbolic geometry

Hyperbolic geometry often has constant negative curvature and is fundamentally different to the Euclidean or spherical geometry which has zero or constant positive curvature, respectively. This enables unique properties such as the divergence of parallel lines and the exponential volume growth towards the boundary. Hyperbolic geometry has five different analytic models to build the hyperbolic space [29]. In this work, we utilize the classical Poincaré ball model as its effectiveness in capturing hierarchical structures [33], [48], [55], [56].

3.2 The Poincaré Ball Model

The Poincaré ball model $(\mathbb{D}_c^n, g^{\mathbb{D}_c})$ with radius $1/\sqrt{c}$ and negative curvature $-c(c > 0)$ is a Riemannian manifold $\mathbb{D}_c^n := \{\mathbf{x} \in \mathbb{R}^n : c\|\mathbf{x}\| < 1\}$ with the metric $g^{\mathbb{D}_c} = \lambda_{c,\mathbf{x}}^2 g^E$, where $\lambda_{c,\mathbf{x}} = \frac{2}{1-c\|\mathbf{x}\|^2}$ is the conformal factor, and $g^E = \mathbf{I}_n$ is the Euclidean metric tensor. The vectorial structure operations defined in Euclidean space can not be directly used in hyperbolic space due to the differential of the space properties. Therefore, we introduce the framework of gyrovector spaces [57], [58] to provide non-associative algebraic operations for hyperbolic space. According to special relativity, when adding speed vectors belonging to the Poincaré ball of radius c , these added speed vectors remain in the ball by the gyrovector spaces operation, also known as the Möbius operation. For instance, the addition and multiplication operations are defined as follows:

Möbius addition operation. For $\mathbf{x}, \mathbf{y} \in \mathbb{D}_c^n$, the Möbius addition operation is defined as:

$$\mathbf{x} \oplus_c \mathbf{y} := \frac{(1 + 2c\langle \mathbf{x}, \mathbf{y} \rangle + c\|\mathbf{y}\|^2)\mathbf{x} + (1 - c\|\mathbf{x}\|^2)\mathbf{y}}{1 + 2c\langle \mathbf{x}, \mathbf{y} \rangle + c^2\|\mathbf{x}\|^2\|\mathbf{y}\|^2}. \quad (1)$$

Möbius scalar multiplication operation. For a scalar $r \in \mathbb{R}$ and a vector $\mathbf{x} \in \mathbb{D}_c^n$, the Möbius scalar multiplication operation is defined as:

$$r \otimes_c \mathbf{x} := (1/\sqrt{c})\tanh(r\tanh^{-1}(\sqrt{c}\|\mathbf{x}\|))\frac{\mathbf{x}}{\|\mathbf{x}\|}. \quad (2)$$

The operation satisfies $r \otimes_c \mathbf{0} := \mathbf{0}$, $r \otimes_c \mathbf{x} = \mathbf{x} \oplus_c \dots \oplus_c \mathbf{x}$ (r additions), $(r' + r) \otimes_c \mathbf{x} = r \otimes_c \mathbf{x} \oplus_c r' \otimes_c \mathbf{x}$, $(r'r) \otimes_c \mathbf{x} = r \otimes_c (r' \otimes_c \mathbf{x})$ and $(|r| \otimes_c \mathbf{x})/\|r \otimes_c \mathbf{x}\| = \mathbf{x}/\|\mathbf{x}\|$.

Möbius matrix multiplication operation. Refer to the relation definition in [50], for a matrix $\mathbf{M} \in \mathbb{R}^{n \times n}$ and a

vector $\mathbf{x} \in \mathbb{D}_c^n$, if $\mathbf{M}\mathbf{x} \neq \mathbf{0}$, the Möbius matrix multiplication operation is defined as:

$$\mathbf{M} \otimes_c \mathbf{x} := \left(\frac{1}{\sqrt{c}} \right) \tanh \left(\frac{\|\mathbf{M}\mathbf{x}\|}{\|\mathbf{x}\|} \tanh^{-1}(\sqrt{c}\|\mathbf{x}\|) \right) \frac{\mathbf{M}\mathbf{x}}{\|\mathbf{M}\mathbf{x}\|}.$$

The distance of $\mathbf{x}, \mathbf{y} \in \mathbb{D}_c^n$ is defined as:

$$d_c^{\mathbb{D}}(\mathbf{x}, \mathbf{y}) = (2/\sqrt{c}) \tanh^{-1}(\sqrt{c}\|\mathbf{x} \oplus_c \mathbf{y}\|). \quad (3)$$

The tangent space $\mathcal{T}_{\mathbf{x}}^c \mathbb{D}_c^n$ at a point $\mathbf{x} \in \mathbb{D}_c^n$ is the first order approximation of $\mathbb{D}_c^n$, which is an n -dimensional Euclidean space. The tangent space $\mathcal{T}_{\mathbf{x}}^c \mathbb{D}_c^n$ and $\mathbb{D}_c^n$ are mapped to each other by exponential ($\mathcal{T}_{\mathbf{x}}^c \mathbb{D}_c^n \mapsto \mathbb{D}_c^n : \exp_{\mathbf{x}}^{\mathbb{D},c}(\cdot)$) and logarithmic ($\mathbb{D}_c^n \mapsto \mathcal{T}_{\mathbf{x}}^c \mathbb{D}_c^n : \log_{\mathbf{x}}^{\mathbb{D},c}(\cdot)$) maps, respectively. For any $\mathbf{x}, \mathbf{y} \in \mathbb{D}_c^n$ and $\mathbf{v} \in \mathcal{T}_{\mathbf{x}}^c \mathbb{D}_c^n$, the mapping functions are given for $\mathbf{v} \neq \mathbf{0}$ and $\mathbf{y} \neq \mathbf{x}$ by:

$$\exp_{\mathbf{x}}^{\mathbb{D},c}(\mathbf{v}) = \mathbf{x} \oplus_c \left(\tanh \left(\sqrt{c} \frac{\lambda_{c,\mathbf{x}} \|\mathbf{v}\|}{2} \right) \frac{\mathbf{v}}{\sqrt{c}\|\mathbf{v}\|} \right), \quad (4)$$

$$\log_{\mathbf{x}}^{\mathbb{D},c}(\mathbf{y}) = \frac{2 \cdot \tanh^{-1}(\sqrt{c}\|\mathbf{x} \oplus_c \mathbf{y}\|)}{\sqrt{c}\lambda_{c,\mathbf{x}}} \frac{-\mathbf{x} \oplus_c \mathbf{y}}{\|\mathbf{x} \oplus_c \mathbf{y}\|}. \quad (5)$$

3.3 Tensor Product

In this section, we introduce the fundamental concept to achieve dual cross relation communication (Sec. 4.2): tensor product. A p -order tensor is indexed by p indices and can be represented as a multidimensional array of data. Formally, a p -order tensor $\mathcal{A}$ can be written as $\mathcal{A} = (a_{i_1, i_2, \dots, i_p}) \in \mathbb{R}^{n_1 \times n_2 \times \dots \times n_p}$. Slices of a tensor are matrices defined from the tensor by holding all but two indices constant. For a 3-order tensor, $\mathcal{A}(:, :, k)$ corresponds the k^{th} frontal slice. For p -order tensors, matrix slices of p -order tensors can be referenced using linear indexing by reshaping the tensor into an $n_1 \times n_2 \times n_3 n_4 \dots n_p$ 3-order tensor and referring to the k^{th} frontal slice as $\mathcal{A}(:, :, k)$. $\mathcal{A}_i \in \mathbb{R}^{n_1 \times n_2 \times \dots \times n_{p-1}}$ for $i = 1, \dots, n_p$ denotes the $(p-1)$ -order tensor created by holding the p th index of $\mathcal{A}$ fixed at i . It is possible to create a tensor in a block circulant pattern, where each block is a tensor of $(p-1)$ -order:

$$\text{circ}(\mathcal{A}) = \begin{bmatrix} \mathcal{A}_1 & \mathcal{A}_{n_p} & \mathcal{A}_{n_p-1} & \dots & \mathcal{A}_2 \\ \mathcal{A}_2 & \mathcal{A}_1 & \mathcal{A}_{n_p} & \dots & \mathcal{A}_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \mathcal{A}_{n_p} & \mathcal{A}_{n_p-1} & \mathcal{A}_{n_p-2} & \dots & \mathcal{A}_1 \end{bmatrix}, \quad (6)$$

where $\text{circ}(\cdot)$ creates a block circulant tensor and the size of $\text{circ}(\mathcal{A})$ is $(n_1 n_p \times n_2 n_p \times \dots \times n_{p-2} n_p \times n_{p-1})$. Although the layout of $\text{circ}(\mathcal{A})$ resembles that of a standard circulant matrix, each entry $\mathcal{A}_i$ in the grid is a $(p-1)$ -order tensor slice $\mathcal{A}_i \in \mathbb{R}^{n_1 \times n_2 \times \dots \times n_{p-1}}$, not a scalar. The outer $n_p \times n_p$ grid encodes the cyclic shifting pattern, while the inner structure of each block retains the full tensor dimensionality. The subsequent $\text{unfold}(\cdot)$ and $\text{fold}(\cdot)$ operations stack and recover these tensor slices along the p -th dimension, respectively. We define $\text{unfold}(\cdot)$ to take an $n_1 \times \dots \times n_p$ tensor $\mathcal{A}$ and return an $n_1 n_p \times n_2 \times \dots \times n_{p-1}$ block tensor in the following way:

$$\text{unfold}(\mathcal{A}) = [\mathcal{A}_1 \quad \mathcal{A}_2 \quad \dots \quad \mathcal{A}_{n_p}]^T. \quad (7)$$

The operation that takes unfold back to tensor form is the “fold” command. Specially, $\text{fold}(\cdot, n_p)$ takes an $n_1 n_p \times n_2 \times \dots \times n_{p-1}$

$\dots \times n_{p-1}$ block tensor and returns an $n_1 \times \dots \times n_p$ tensor, defined as:

$$\text{fold}(\text{unfold}(\mathcal{A}), n_p) = \mathcal{A}. \quad (8)$$

Next, we introduce two special tensor product operators, i.e., **3-order T-product** and **Higher-order T-product**.

Definition 3.1 (3-order T-product operator) For $\mathcal{A} \in \mathbb{R}^{n_1 \times n_2 \times n_3}$ and $\mathcal{B} \in \mathbb{R}^{n_2 \times l \times n_3}$, the 3-order T-product $\mathcal{C} \in \mathbb{R}^{n_1 \times l \times n_3} = \mathcal{A} * \mathcal{B}$ is defined as:

$$\mathcal{C} = \mathcal{A} * \mathcal{B} = \text{fold}(\text{circ}(\mathcal{A}) \cdot \text{unfold}(\mathcal{B})), \quad (9)$$

where “ $*$ ” represents a tensor product, “ $\cdot$ ” represents standard matrix product.

Definition 3.2 (Higher-order T-product operator) For $\mathcal{A} \in \mathbb{R}^{n_1 \times n_2 \times n_3 \dots \times n_p}$ and $\mathcal{B} \in \mathbb{R}^{n_2 \times l \times n_3 \dots \times n_p}$, the High-order T-product $\mathcal{C} \in \mathbb{R}^{n_1 \times l \times n_3 \dots \times n_p} = \mathcal{A} * \mathcal{B}$ is defined as:

$$\mathcal{C} = \mathcal{A} * \mathcal{B} = \text{fold}(\text{circ}(\mathcal{A}) * \text{unfold}(\mathcal{B})). \quad (10)$$

4 METHODOLOGY

In this section, we present HyperCLIP++, a novel framework for open-vocabulary semantic segmentation, as shown in Fig. 3. The framework operates by (1) adapting CLIP’s embeddings to the segmentation task through adjustments of their hyperbolic radii within the Poincaré ball model (Sec. 4.1), and (2) aligning CLIP’s visual-textual hierarchy through the proposed Dual Cross-Relation Communication (DCRC) module. This design ensures consistent cross-modal alignment with minimal computational cost (Sec. 4.2). Finally, we provide a summary of the overall framework in Sec. 4.3, followed by a detailed presentation of the theoretical analysis and derivation in Sec. 5.

4.1 Hyperbolic Radius Adjustment

Intuition of hyperbolic radius adjustment. CLIP is pre-trained using image-level supervision, whereas semantic segmentation requires dense, pixel-level representations. When CLIP is adapted to segmentation, its text embeddings, together with the corresponding visual embeddings, consistently shift toward smaller hyperbolic radii. Motivated by this observation, HyperCLIP++ does not learn an unrestricted update to the entire CLIP parameter space. Instead, it learns lightweight block-diagonal transformations that directly and controllably adjust the embedding radii while retaining most of CLIP’s pre-trained parameters.

Generally, to enable training in hyperbolic space, a feature embedding $\mathbf{z}$ in Euclidean space is first mapped to hyperbolic space via the exponential map at a local reference point, typically the origin (e.g., Eq. 4), as adopted in prior work such as hyperLoRA [54]. Within the hyperbolic space, a linear transformation parameterized by a learnable weight matrix $\mathbf{W}$ is applied to obtain the hyperbolic representation. Finally, the result is mapped back to Euclidean space through the logarithmic map (e.g., Eq. 5). Formally, in the Poincaré ball model, this procedure can be expressed as:

$$\mathbf{W}\mathbf{z} = \log_0^{\mathbb{D},c}(\mathbf{W} \otimes_c \exp_0^{\mathbb{D},c}(\mathbf{z})), \quad (11)$$

where $\otimes_c$ is the möbius matrix multiplication operation. The primary aim of previous methods is to capture more intricate hierarchical relationships within hyperbolic space. In

contrast, our goal is to directly adjust the hyperbolic radius of CLIP's embeddings, leading to a scaling transformation.

Fine-tuning with scaling transformation. The objective of hyperbolic radius adjustment is to introduce a scaling transformation to directly adjust the hyperbolic radii of CLIP's embeddings in hyperbolic space, thereby equipping CLIP with segmentation ability. Concretely, it necessitates the utilization of a diagonal matrix, denoted as $\mathbf{S}$, to scale a CLIP's feature embedding $\mathbf{z} \in \mathbb{R}^{b \times d}$, where d is the feature dimension of $\mathbf{z}$, b is the spatial dimension of input images or number of classes. When extending this process to hyperbolic space, $\mathbf{S}$ can directly adjust the hyperbolic radius of $\mathbf{z}$ (the theoretical analysis of this finding can be found in Sec. 5). Specifically, we realize the diagonal matrix $\mathbf{S}$ in a block-wise manner,

$$\mathbf{S} = \text{diag}(\mathbf{S}_1, \mathbf{S}_2, \dots, \mathbf{S}_k, \dots, \mathbf{S}_K), \quad (12)$$

where $\mathbf{S}_k \in \mathbb{R}^{n \times n}$ denotes a square matrix of the k -th block, and the block number K is calculated by $K = d/n$. By adjusting the size of n , we can achieve a transition between naive scaling transformation ($n = 1$, i.e., modifying only the diagonal elements) and arbitrary transformation ($n = d$, i.e., fully fine-tuning in hyperbolic space). The ablation study of n is presented in Sec. 6.3.

Parameter complexity. For an K -block diagonal matrix, the number of parameters is d^2/K , resulting in a complexity of $\mathcal{O}(d^2/K)$. We can optionally share the block matrix to reduce the number of parameters, i.e., $\mathbf{S}_k = \mathbf{S}_j, \forall k \neq j$. This reduces the parameter complexity to $\mathcal{O}(d^2/K^2)$. We can further reduce parameter complexity by setting $n = 1$, which transforms the block-diagonal matrix into a diagonal matrix. In this case, the number of parameters becomes d , resulting in a complexity of $\mathcal{O}(d)$. Despite all these strategies to improve parameter efficiency, we note that the resulting matrix $\mathbf{S}$ remains to realize scaling transformations, thereby allowing it to adjust the radius of CLIP's embeddings.

Geometric Complexity Analysis. Compared to performing the same scaling transformation in Euclidean space, the only additional cost of operating in hyperbolic space is the exponential and logarithmic maps (Eqs. 4–5) for bidirectional projection. At the origin, these maps reduce to element-wise operations ($\tanh$, radius, scalar division), which can be completed in $\mathcal{O}(d)$ per embedding, where d is the embedding dimension. This is negligible compared to the $\mathcal{O}(d^2)$ cost of each Transformer self-attention layer. Empirically, this bidirectional mapping overhead results in less than 1% total latency increase.

4.2 Dual Cross Relation Communication

Intuition of DCRC. Each learnable block-diagonal matrix can be viewed as a local adjustment controller associated with one encoder layer and one modality. DCRC collects these controllers into a unified tensor and enables communication along two dimensions: modality and layer. Communication along the modality dimension coordinates the radius adjustments of the visual and textual embeddings, whereas communication along the layer dimension directly coordinates controllers at different encoder depths. The higher-order tensor formulation below implements this two-dimensional communication efficiently.

While adjusting the hyperbolic radii of embeddings endows CLIP with segmentation capabilities, this hierarchy alignment process is performed separately within CLIP's visual and textual encoders. Such independent optimization, especially with a limited number of learnable parameters, may lead to a risk of misalignment between their hierarchical representations during training. This problem is further exacerbated by two factors: the limitations of the hidden Markov structure across layers [59]–[61], and the nature of CLIP's late-fusion design, which prevents any interaction between modalities until the final output embeddings. One might consider introducing a cross-attention mechanism between text and visual embeddings; however, such designs often disrupt the inherent hierarchical structure of CLIP and introduce excessive parameters, potentially causing overfitting and undermining CLIP's generalization ability. Based on the above analysis, HyperCLIP++ introduces Dual Cross-Relation Communication (DCRC), which facilitates interaction among different layers and modalities (i.e., text and image) in hyperbolic space with negligible parameter overhead. DCRC establishes explicit communication between the vision and text pathways before the final encoder output, and thus prevents misalignment issues during training.

DCRC introduces cross-layer and cross-modality communication among different block-diagonal matrices, i.e., $\mathbf{S}$, achieved through two relation projections. To do this, we first treat all blocks in ℓ^{th} layer as an individual slice in this 3-order tensor $\mathcal{T}_\ell$, which is derived as follows:

$$\mathcal{T}_\ell = [\mathbf{R}_{v\ell 1}, \mathbf{R}_{e\ell 1}, \dots, \mathbf{R}_{v\ell i}, \mathbf{R}_{e\ell i}, \dots, \mathbf{R}_{v\ell b}, \mathbf{R}_{e\ell b}] \in \mathbb{R}^{q \times q \times (b+b)}, \quad (13)$$

where $q = d/b$. Then, we treat the tensor $\mathcal{T}_\ell$ as an individual slice within a 4-order tensor $\mathcal{T}$, defined as follows:

$$\mathcal{T} = [\mathcal{T}_1, \mathcal{T}_2, \dots, \mathcal{T}_\ell, \dots, \mathcal{T}_L] \in \mathbb{R}^{q \times q \times (b+b) \times L}. \quad (14)$$

Initially, according to the characteristics of gradient propagation in deep learning theory, i.e., chain rule, each frontal slice $\mathbf{R}_{\ell i} \in \{\mathbb{R}^{q \times q}\}^{(b+b) \times L}$ is updated sequentially in CLIP's encoder. As a result, updating the $\mathcal{T}$ lacks cross-frontal-slice communication, limiting the flexibility of adjusting fine-tuned projection. To explore efficient communication methods between the frontal slices $\mathbf{R}_{\ell i}$ in higher-order tensor $\mathcal{T}$, we reanalyze the two special tensor product operators.

For $\mathcal{A} \in \mathbb{R}^{n_1 \times n_2 \times n_3}$, according to the **3-order T-product**, there is an invertible transform $S_3(\cdot) : \mathbb{R}^{n_1 \times n_2 \times n_3} \rightarrow \mathbb{R}^{n_1 \times n_2 \times n_3}$ in third dimension and it transform the Eq. (9) as:

$$\begin{aligned} \mathcal{C} &= S_3^{-1}(S_3(\mathcal{A}) \odot S_3(\mathcal{B})) \\ &= S_3^{-1}(\bar{\mathcal{A}} \odot \bar{\mathcal{B}}) = S_3^{-1}(\bar{\mathcal{C}}), \end{aligned} \quad (15)$$

where $\bar{\mathcal{C}} = \bar{\mathcal{A}} \odot \bar{\mathcal{B}}$ denotes the frontal-slice-wise product (Definition 5.1 refers to [62]) $\bar{\mathcal{C}}(;;, i) = \bar{\mathcal{A}}(;;, i) \cdot \bar{\mathcal{B}}(;;, i), i = 1, 2, \dots, n_3$ and $S_3^{-1}(\cdot)$ is the inverse transform of $S_3(\cdot)$. According to the definition of the frontal-slice-wise product, the invertible transform $S_3(\cdot)$ is formulated as:

$$\bar{\mathcal{A}} = S_3(\mathcal{A}) = \mathcal{A} \times_3 \mathbf{S}_3, \quad (16)$$

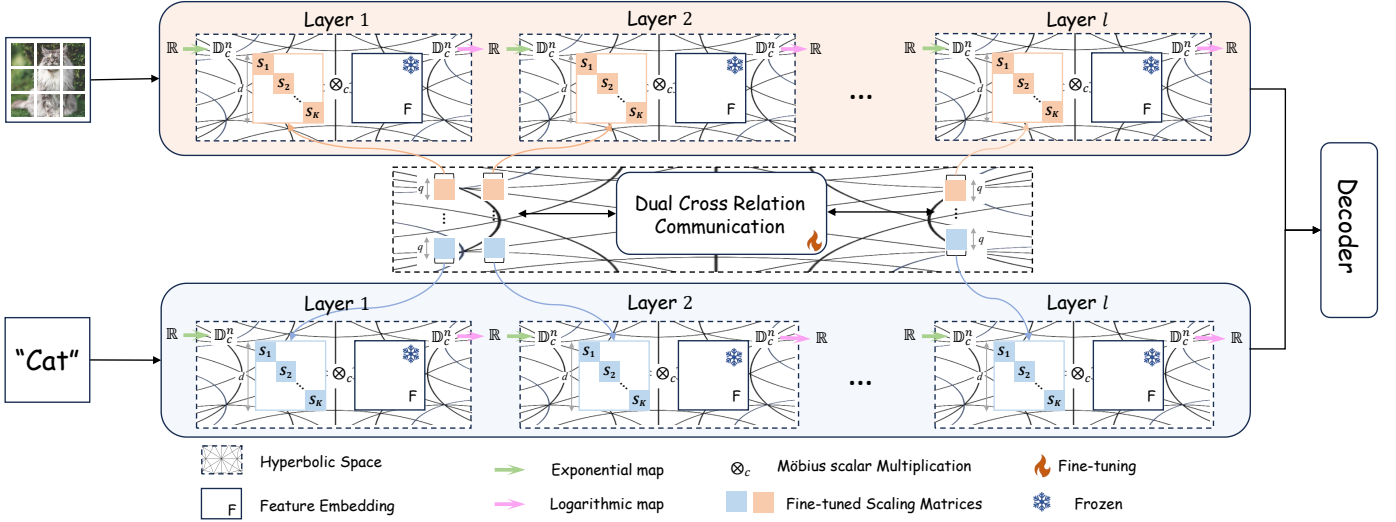


Fig. 3: **A schematic representation of HyperCLIP++.** In the HyperCLIP++ framework, we fine-tune CLIP in hyperbolic space by minimizing its embeddings' hyperbolic radii using several tunable block-diagonal scaling matrices. Then, we introduce a Dual Cross-Relation Communication (DCRC) module that operates on all tunable block-diagonal scaling matrices to promote cross-layer and cross-modality interactions. This module facilitates hierarchy alignment between the vision and text modalities of CLIP.

where “ $\times_3$ ” denotes the mode-3 product and $\mathbf{S}_3 \in \mathbb{R}^{n_3 \times n_3}$ is an arbitrary invertible matrix. Similarly, the inverse transform of Eq. (16) is derived as:

$$\mathcal{A} = \mathbf{S}_3^{-1}(\bar{\mathcal{A}}) = \bar{\mathcal{A}} \times_3 \mathbf{S}_3^{-1}. \quad (17)$$

Similarly, for $\mathcal{A} \in \mathbb{R}^{n_1 \times n_2 \times \dots \times n_p}$, according to the **Higher-order T-product**, there are invertible transform $S_i(\cdot) : \mathbb{R}^{n_1 \times n_2 \times \dots \times n_p} \rightarrow \mathbb{R}^{n_1 \times n_2 \times \dots \times n_p}, i = 3, 4, \dots, p$ in i^{th} dimension and they transform the Eq. (10) as:

$$\mathcal{C} = \tilde{S}^{-1}(\tilde{S}(\mathcal{A}) \odot \tilde{S}(\mathcal{B})) = \tilde{S}^{-1}(\bar{\mathcal{A}} \odot \bar{\mathcal{B}}) = \tilde{S}^{-1}(\bar{\mathcal{C}}), \quad (18)$$

where $\tilde{S}(\mathcal{A}) = S_p(S_{p-1}(\dots S_3(\mathcal{A}) \dots))$, $\bar{\mathcal{C}} = \bar{\mathcal{A}} \odot \bar{\mathcal{B}}$ denotes the frontal-slice-wise product $\bar{\mathcal{C}}(i, :, i) = \bar{\mathcal{A}}(i, :, i) \cdot \bar{\mathcal{B}}(i, :, i), i = 1, 2, \dots, n_3 n_4 \dots n_p$ and $\tilde{S}^{-1}(\cdot)$ is the inverse transform of $\tilde{S}(\cdot)$. Similarly, the inverse transform $\tilde{S}(\cdot)$ is formulated as:

$$\bar{\mathcal{A}} = \tilde{S}(\mathcal{A}) = \mathcal{A} \times_3 \mathbf{S}_3 \times_4 \mathbf{S}_4 \dots \times_p \mathbf{S}_p, \quad (19)$$

and its inverse transform is derived as:

$$\mathcal{A} = \tilde{S}^{-1}(\bar{\mathcal{A}}) = \bar{\mathcal{A}} \times_3 \mathbf{S}_3^{-1} \times_4 \mathbf{S}_4^{-1} \dots \times_p \mathbf{S}_p^{-1}. \quad (20)$$

Derivation. please refer to Sec. 5. ■

According to Eqs. (40), (41) and (42), we adopt its idea and design arbitrary invertible relation matrix $\mathbf{S}_3 \in \mathbb{R}^{(b+b) \times (b+b)}$ and $\mathbf{S}_4 \in \mathbb{R}^{L \times L}$ to capture the cross-modality and cross-layer information in $\mathcal{T}$. Then the updated tensor $\mathcal{T}_w$ is formulated as:

$$\mathcal{T}_w = \mathcal{T} \times_3 \mathbf{S}_3 \times_4 \mathbf{S}_4 \in \mathbb{R}^{q \times q \times (b+b) \times L}, \quad (21)$$

where the relation matrix $\mathbf{S}_3$ and $\mathbf{S}_4$ are learnable. To better capture the nonlinear interactions inside the whole parameter space, we further adopt an MLP of k layers $f_3(\cdot)$ and $f_4(\cdot)$ to replace the transform $\times_3 \mathbf{S}_3$ and $\times_4 \mathbf{S}_4$, respectively, and the MLP $f_3(\cdot)$ is formulated as:

$$f_3(\mathcal{T}) = \sigma(\dots \sigma(\mathcal{A} \times_3 \mathbf{W}_1) \dots) \times_3 \mathbf{W}_k, \quad (22)$$

where $\sigma(\cdot)$ is a nonlinear scalar function and matrices $\{\mathbf{W}_j \in \mathbb{R}^{(b+b) \times (b+b)}\}_{j=1}^k$. The MLP $f_4(\cdot)$ is similar. Finally, the $\mathcal{T}$ is updated by $\mathcal{T} = \mathcal{T} + \alpha \mathcal{T}_w$, where $\alpha \in \mathbb{R}^{(b+b) \times L}$ is a learnable parameter.

4.3 Overall Architecture

Matrix-wise Transformation. During fine-tuning, hyperbolic radius adjustment introduces a scaling transformation into CLIP's encoder. For a feature embedding $\mathbf{z}$, the forward pass is defined as:

$$\mathbf{S}\mathbf{z} = \log^{\mathbb{D},c} \mathbf{0}(\mathbf{S} \otimes_c \exp^{\mathbb{D},c} \mathbf{0}(\mathbf{z})), \quad (23)$$

where $\mathbf{S}$ denotes a block-diagonal scaling matrix in hyperbolic space. Meanwhile, the original CLIP components load pre-trained weights, whose parameters remain frozen.

Network-wise Parameter Coordination. To ensure global coherence across individual transformations, we represent the collection of all learnable matrices $\mathbf{S}$ in the ℓ -th layer as a third-order tensor $\mathcal{T}_\ell$. Aggregating across layers yields a unified fourth-order parameter tensor for the entire network:

$$\mathcal{T} = [\mathcal{T}_1, \mathcal{T}_2, \dots, \mathcal{T}_\ell, \dots, \mathcal{T}_L]. \quad (24)$$

This network-level tensor is then updated by the DCRC module:

$$\mathcal{T}_w = \text{DCRC}(\mathcal{T}), \quad (25)$$

where $\text{DCRC}(\cdot)$ denotes the global coordination mechanism described in Sec. 4.2, synchronizing parameters across both modalities and layers. This operation encapsulates the coordinated effect of all learnable matrices in the CLIP's encoder. **Loss Function.** Following previous works [22], [23], we incorporate a classical cross-entropy loss, denoted as $\mathcal{L}_{ce}$, for the fine-tuning of CLIP.

5 THEORETICAL ANALYSIS AND DERIVATION

5.1 Hyperbolic Radius Adjustment via Scaling

Theorem 1 (Scaling). According to the definitions of the classical Poincaré Ball model and the hyperbolic tangent, for a point $\mathbf{x} \in \mathbb{D}_c^n$ in hyperbolic space, its hyperbolic radius (i.e., hyperbolic induced norm) is defined as:

$$\text{Rad}_{\mathbf{x}} := d_c^{\mathbb{D}}(\mathbf{x}, \mathbf{0}) = \left(\frac{2}{\sqrt{c}}\right) \tanh^{-1}(\sqrt{c}\|\mathbf{x}\|), \quad (26)$$

where $\mathbf{0}$ is the center of the hyperbolic space. Let s be a scaling parameter, combine with the Möbius scalar multiplication operation described in Section 3.2 and Eq. 26, the radius of the point $\mathbf{x}_s = \mathcal{T}_{\mathbf{x}}(s) := s \otimes_c \mathbf{x}$ which changed by the scaling transformation $\mathcal{T}_{\mathbf{x}}(s)$ is obtained by:

$$\begin{aligned} \text{Rad}_{\mathbf{x},s}(s) &= \frac{2}{\sqrt{c}} \tanh^{-1}(\sqrt{c}\|\mathbf{x}_s\|) \\ &= \frac{2}{\sqrt{c}} \tanh^{-1}\left(\sqrt{c}\left\|\frac{\tanh\left(\frac{s\sqrt{c}}{2}\text{Rad}_{\mathbf{x}}\right)\mathbf{x}}{\sqrt{c}\|\mathbf{x}\|}\right\|\right) \\ &= s\text{Rad}_{\mathbf{x}}. \end{aligned} \quad (27)$$

Notably, $\text{Rad}_{\mathbf{x},s}(s)$ satisfies the monotonicity property with respect to s , allowing the hyperbolic radius to be adjusted by varying s . Furthermore, according to the Möbius matrix multiplication operation defined in [50], a scaling matrix $\mathbf{S}$ can similarly modify the hyperbolic radius through the transformation $\mathcal{T}_{\mathbf{x}}(\mathbf{S})$.

Proof. In a hyperbolic space, considering a point $\mathbf{x} \in \mathbb{D}_c^n$ with hyperbolic radius $\text{Rad}_{\mathbf{x}} = (2/\sqrt{c}) \tanh^{-1}(\sqrt{c}\|\mathbf{x}\|)$, we scale the hyperbolic radius $\text{Rad}_{\mathbf{x}}$ by utilizing the Möbius scalar multiplication operation to change the point $\mathbf{x}$ by a scaling parameter s , which can be formulated as:

$$\mathbf{x}_s = \mathcal{T}_{\mathbf{x}}(s) := s \otimes_c \mathbf{x}. \quad (28)$$

The hyperbolic radius $\text{Rad}_{\mathbf{x},s}(s)$ of the point $\mathbf{x}_s$ is obtained:

$$\begin{aligned} \text{Rad}_{\mathbf{x},s}(s) &= \frac{2}{\sqrt{c}} \tanh^{-1}(\sqrt{c}\|\mathbf{x}_s\|) \\ &= \frac{2}{\sqrt{c}} \tanh^{-1}\left(\sqrt{c}\left\|\frac{\tanh\left(s \tanh^{-1}(\sqrt{c}\|\mathbf{x}\|)\right)\mathbf{x}}{\sqrt{c}\|\mathbf{x}\|}\right\|\right) \\ &= \frac{2}{\sqrt{c}} \tanh^{-1}\left(\sqrt{c}\left\|\frac{\tanh\left(s \frac{\sqrt{c}}{2}\text{Rad}_{\mathbf{x}}\right)\mathbf{x}}{\sqrt{c}\|\mathbf{x}\|}\right\|\right) \\ &= \frac{2}{\sqrt{c}} \tanh^{-1}\left(\left\|\tanh\left(s \frac{\sqrt{c}}{2}\text{Rad}_{\mathbf{x}}\right) \frac{\mathbf{x}}{\|\mathbf{x}\|}\right\|\right) \\ &= \frac{2}{\sqrt{c}} \tanh^{-1}\left(\frac{\tanh\left(s \frac{\sqrt{c}}{2}\text{Rad}_{\mathbf{x}}\right)}{\|\mathbf{x}\|}\|\mathbf{x}\|\right) \\ &= \frac{2}{\sqrt{c}} \tanh^{-1}\left(\tanh\left(s \frac{\sqrt{c}}{2}\text{Rad}_{\mathbf{x}}\right)\right) \\ &= s\text{Rad}_{\mathbf{x}}. \end{aligned} \quad (29)$$

The $\text{Rad}_{\mathbf{x},s}(s)$ satisfies the monotonicity criteria of s . For a scaling matrix $\mathbf{S}$, according to the relation definition of

Möbius matrix multiplication in [50], the hyperbolic radius $\text{Rad}_{\mathbf{x},\mathbf{S}}(\mathbf{S})$ is obtained as:

$$\begin{aligned} \text{Rad}_{\mathbf{x},\mathbf{S}}(\mathbf{S}) &= \frac{2}{\sqrt{c}} \tanh^{-1}(\sqrt{c}\|\mathcal{T}_{\mathbf{x}}(\mathbf{S})\|) \\ &= \frac{2}{\sqrt{c}} \tanh^{-1}(\sqrt{c}\|\mathbf{S} \otimes_c \mathbf{x}\|) \\ &= \frac{2}{\sqrt{c}} \tanh^{-1}\left(\sqrt{c}\left\|\frac{\tanh\left(\frac{\|\mathbf{S}\mathbf{x}\|}{\|\mathbf{x}\|} \tanh^{-1}(\sqrt{c}\|\mathbf{x}\|)\right)\mathbf{S}\mathbf{x}}{\sqrt{c}\|\mathbf{S}\mathbf{x}\|}\right\|\right) \\ &= \frac{2}{\sqrt{c}} \tanh^{-1}\left(\sqrt{c}\left\|\frac{\tanh\left(\frac{\|\mathbf{S}\mathbf{x}\|}{\|\mathbf{x}\|} \cdot \frac{\sqrt{c}}{2}\text{Rad}_{\mathbf{x}}\right)\mathbf{S}\mathbf{x}}{\sqrt{c}\|\mathbf{S}\mathbf{x}\|}\right\|\right) \\ &= \frac{2}{\sqrt{c}} \tanh^{-1}\left(\left\|\tanh\left(\frac{\|\mathbf{S}\mathbf{x}\|}{\|\mathbf{x}\|} \frac{\sqrt{c}}{2}\text{Rad}_{\mathbf{x}}\right) \frac{\mathbf{S}\mathbf{x}}{\|\mathbf{S}\mathbf{x}\|}\right\|\right) \\ &= \frac{2}{\sqrt{c}} \tanh^{-1}\left(\frac{\tanh\left(\frac{\|\mathbf{S}\mathbf{x}\|}{\|\mathbf{x}\|} \frac{\sqrt{c}}{2}\text{Rad}_{\mathbf{x}}\right)}{\|\mathbf{S}\mathbf{x}\|}\|\mathbf{S}\mathbf{x}\|\right) \\ &= \frac{2}{\sqrt{c}} \tanh^{-1}\left(\tanh\left(\frac{\|\mathbf{S}\mathbf{x}\|}{\|\mathbf{x}\|} \frac{\sqrt{c}}{2}\text{Rad}_{\mathbf{x}}\right)\right) \\ &= \frac{\|\mathbf{S}\mathbf{x}\|}{\|\mathbf{x}\|} \text{Rad}_{\mathbf{x}}. \end{aligned} \quad (30)$$

It is noticed that the $\text{Rad}_{\mathbf{x},s}(s)$ satisfies the monotonicity criteria, therefore, we can adjust the hyperbolic radius by changing the scaling parameter s . Refer to the definition of the Möbius matrix multiplication operation in [50], the scaling matrix $\mathbf{S}$ also has the ability to adjust the hyperbolic radius via the scaling transformation $\mathcal{T}_{\mathbf{x}}(\mathbf{S})$. ■

5.2 DCRC via Higher-order T-product

Our DCRC module organizes the learnable scaling matrices into a 4th-order tensor $\mathcal{T} \in \mathbb{R}^{q \times q \times (b+b) \times L}$. Standard tensor operations and backpropagation updates are typically confined to individual “frontal slices” (the $q \times q$ matrices), lacking a mechanism for efficient communication across the higher-order dimensions (modality and layer). The Higher-order T-product provides an elegant solution.

Derivation 1. According to [63], if $\mathcal{A}$ is $n_1 \times n_2 \times n_3$, $\mathcal{A}$ can be block diagonalized by using Discrete Fourier Transformer (DFT) matrix $\mathbf{F}_{n_3} \in \mathbb{R}^{n_3 \times n_3}$ as:

$$\begin{aligned} &(\mathbf{F}_{n_3} \otimes \mathbf{I}_{n_1}) \cdot \text{circ}(\text{unfold}(\mathcal{A})) \cdot (\mathbf{F}_{n_3}^* \otimes \mathbf{I}_{n_2}) \\ &= \mathbf{D} = \begin{bmatrix} \mathbf{D}_1 & & \\ & \ddots & \\ & & \mathbf{D}_{n_3} \end{bmatrix} \in \mathbb{R}^{n_1 n_3 \times n_2 n_3}, \end{aligned} \quad (31)$$

where “ $\otimes$ ” denotes the Kronecker product, “ $\mathbf{F}_{n_3}^*$ ” denotes the conjugate transpose of $\mathbf{F}_{n_3}$, “ $\cdot$ ” means standard matrix product and $\mathbf{D}$ is a block diagonal matrix. In fact, the i -th block matrix $\mathbf{D}_i$ of $\mathbf{D}$ can be computed by applying DFT of $\mathcal{A}$ along 3-rd dimension. The **3-order T-product** in Eq. (15) can be computed as:

$$\begin{aligned} &(\mathbf{F}_{n_3}^* \otimes \mathbf{I}_{n_1}) \cdot ((\mathbf{F}_{n_3} \otimes \mathbf{I}_{n_1}) \cdot \text{circ}(\text{unfold}(\mathcal{A})) \\ &\quad \cdot (\mathbf{F}_{n_3}^* \otimes \mathbf{I}_{n_2})) \cdot (\mathbf{F}_{n_3} \otimes \mathbf{I}_{n_2}) \cdot \text{unfold}(\mathcal{B}). \end{aligned} \quad (32)$$

It is readily shown that $(\mathbf{F}_{n_3} \otimes \mathbf{I}_{n_2})$ unfold can be computed by applying DFT of $\mathcal{B}$ along 3-rd dimension: the result called $\tilde{\mathbf{B}}$. Thus, Eq. (32) remains to multiply each block matrix $\mathbf{D}_i$ of $\mathbf{D}$ with each block matrix $\mathbf{B}_i$ of $\tilde{\mathbf{B}}$, then take an inverse DFT along the 3-rd dimension of the result. Hence, the **3-order T-product** in Eq. (15) can be re-formulated as:

$$\begin{aligned} \mathcal{C} &= \text{DFT}_3^{-1}(\text{DFT}_3(\mathcal{A}) \odot \text{DFT}_3(\mathcal{B})) \\ &= \text{DFT}_3^{-1}(\tilde{\mathcal{A}} \odot \tilde{\mathcal{B}}) = \text{DFT}_3^{-1}(\tilde{\mathcal{C}}), \end{aligned} \quad (33)$$

where $\text{DFT}_3(\cdot)$ is DFT along 3-rd dimension and $\text{DFT}_3^{-1}(\cdot)$ is the inverse DFT along 3-rd dimension. In mathematics, the DFT of $\mathcal{A}$ along 3-rd dimension is formulated as:

$$\tilde{\mathcal{A}} = \text{DFT}_3(\mathcal{A}) = \mathcal{A} \times_3 \mathbf{F}_{n_3}. \quad (34)$$

Similarly, the inverse DFT of $\tilde{\mathcal{A}}$ along 3-rd dimension is derived as:

$$\mathcal{A} = \text{DFT}_3^{-1}(\tilde{\mathcal{A}}) = \tilde{\mathcal{A}} \times_3 \mathbf{F}_{n_3}^{-1}. \quad (35)$$

By the detailed theoretical analysis in [64], the DFT has been extended to a general invertible transform S with an invertible transform matrix $\mathbf{S}$. In mathematics, the invertible transform of $\mathcal{A}$ along 3-rd dimension is formulated as:

$$\tilde{\mathcal{A}} = S_3(\mathcal{A}) = \mathcal{A} \times_3 \mathbf{S}_{n_3}. \quad (36)$$

Similarly, the inverse transform of $\tilde{\mathcal{A}}$ along 3-rd dimension is derived as:

$$\mathcal{A} = S_3^{-1}(\tilde{\mathcal{A}}) = \tilde{\mathcal{A}} \times_3 \mathbf{S}_{n_3}^{-1}. \quad (37)$$

Similarly, if $\mathcal{A} \in \mathbb{R}^{n_1 \times n_2 \times \dots \times n_p}$, $\mathcal{A}$ can be block diagonalized by using a sequence of DFT matrices $\mathbf{F}_{n_i} \in \mathbb{R}^{n_i \times n_i}$, $i = 3, 4, \dots, p$ as:

$$\begin{aligned} &(\mathbf{F}_{n_p} \otimes \mathbf{F}_{n_{p-1}} \otimes \dots \otimes \mathbf{F}_{n_3} \otimes \mathbf{I}_{n_1}) \cdot \tilde{\mathcal{A}} \\ &\cdot (\mathbf{F}_{n_p}^* \otimes \mathbf{F}_{n_{p-1}}^* \otimes \dots \otimes \mathbf{F}_{n_3}^* \otimes \mathbf{I}_{n_2}) = \mathbf{D}, \end{aligned} \quad (38)$$

where $\tilde{\mathcal{A}} = \text{circ}(\text{unfold}(\mathcal{A})) \in \mathbb{R}^{n_1 n_3 n_4 \dots n_p \times n_2 n_3 \dots n_p}$. Since the matrix $\mathbf{D}$ is block diagonal with $n_3 n_4 \dots n_p$ blocks each of size $n_1 \times n_2$, the **Higher-order T-product** in Eq. (16) can be computed as:

$$(\tilde{\mathbf{F}}^* \otimes \mathbf{I}_{n_1}) \cdot ((\tilde{\mathbf{F}} \otimes \mathbf{I}_{n_1}) \cdot \tilde{\mathcal{A}} \cdot (\tilde{\mathbf{F}}^* \otimes \mathbf{I}_{n_2})) \cdot (\tilde{\mathbf{F}}_{n_3} \otimes \mathbf{I}_{n_2}) \cdot \tilde{\mathcal{B}}, \quad (39)$$

where $\tilde{\mathbf{F}} = \mathbf{F}_{n_p} \otimes \mathbf{F}_{n_{p-1}} \otimes \dots \otimes \mathbf{F}_{n_3}$. Using the DEF, it is straightforward to show that the block diagonal matrix $\mathbf{D}$ in Eq. (38) can be obtained by repeating DFTs of $\mathcal{A}$ along each dimension expect for 1-st and 2-nd dimension. Similarly, by using a sequence invertible transform $S_j(\cdot)$, $i = 3, 4, \dots, p$ with invertible transform matrix $\mathbf{S}_i$, the **Higher-order T-product** in Eq. (10) can be re-formulated as:

$$\mathcal{C} = \tilde{S}^{-1}(\tilde{S}(\mathcal{A}) \odot \tilde{S}(\mathcal{B})) = \tilde{S}^{-1}(\tilde{\mathcal{A}} \odot \tilde{\mathcal{B}}) = \tilde{S}^{-1}(\tilde{\mathcal{C}}), \quad (40)$$

where $\tilde{S}(\mathcal{A}) = S_p(S_{p-1}(\dots S_3(\mathcal{A}) \dots))$, $\tilde{\mathcal{C}} = \tilde{\mathcal{A}} \odot \tilde{\mathcal{B}}$ denotes the frontal-slice-wise product $\tilde{\mathcal{C}}(\cdot, \cdot, \cdot, i) = \tilde{\mathcal{A}}(\cdot, \cdot, \cdot, i) \cdot \tilde{\mathcal{B}}(\cdot, \cdot, \cdot, i)$, $i = 1, 2, \dots, n_3 n_4 \dots n_p$ and $\tilde{S}^{-1}(\cdot)$ is the inverse transform of $\tilde{S}(\cdot)$. The inverse transform $\tilde{S}(\cdot)$ is formulated as:

$$\tilde{\mathcal{A}} = \tilde{S}(\mathcal{A}) = \mathcal{A} \times_3 \mathbf{S}_3 \times_4 \mathbf{S}_4 \dots \times_p \mathbf{S}_p, \quad (41)$$

and its inverse transform is derived as:

$$\mathcal{A} = \tilde{S}^{-1}(\tilde{\mathcal{A}}) = \tilde{\mathcal{A}} \times_3 \mathbf{S}_3^{-1} \times_4 \mathbf{S}_4^{-1} \dots \times_p \mathbf{S}_p^{-1}. \quad (42)$$

6 EXPERIMENTS

6.1 Experimental Setup

Datasets. Following prior works [22], [23], we use the COCO-Stuff dataset [25] as our training set. This dataset consists of around 118,000 densely annotated images, covering 171 distinct semantic categories. For inference, we compare our method against state-of-the-art approaches across several semantic segmentation benchmarks, including ADE20K [67], PASCAL VOC [68], and PASCAL-Context [69].

- **ADE20K** [67] is a well-established semantic segmentation dataset with approximately 20,000 training images and 2,000 validation images. It also includes two separate test sets: A-150, consisting of 150 common categories, and A-847, which contains 847 categories.
- **PASCAL VOC** [68] is a smaller dataset for semantic segmentation, consisting of 1464 training images and 1449 validation images, covering 20 different foreground categories. We refer to this dataset as PAS-20. In line with [23], we also report scores on PAS-20^b, which includes the “background” as the 21st category.
- **PASCAL-Context** [69] is an extended version of the original PASCAL VOC dataset. It provides two test sets: PC-59 and PC-459, containing 59 and 459 categories respectively, for evaluation purposes.

Evaluation Metric. Consistent with prior studies [22], [23], we use mean Intersection over Union (mIoU) to evaluate the semantic segmentation performance on these benchmarks.

Implementation Details. We implement our method using the Transformer-based CLIP model. Following the protocol established in [23], we mainly evaluate our results on two versions of the CLIP model: ViT-B/16 and ViT-L/14. For training, we follow the same experimental setup as [23]. Besides, we use the Adam optimizer [70] with an initial learning rate of 1×10^{-5} for CLIP, and a weight decay of 10^{-4} . Training is conducted with one image per mini-batch. We apply the block-diagonal learnable weight in the self-attention module in each layer of CLIP. The MLP in dual cross-relation communication module is realized by a standard FFN. We employ the cost-based approach from [23] as our decoder. The curvature c of Poincaré ball model is set to 0.01. All models are trained for 80,000 iterations using 4 NVIDIA A800 GPUs.

6.2 Comparisons with State-of-the-art Methods

Here, we compare our proposed method with several state-of-the-art methods, as shown in Table 1, using six test sets across three benchmarks. Overall, we achieve the best results. Most existing open-vocabulary semantic segmentation methods follow partial fine-tuning strategies, i.e., fine-tuning CLIP’s image encoder. Although these methods provide sufficient flexibility for aligning with text descriptions generated by the text encoder, they fail to adjust the text embeddings to the appropriate hierarchical level for effective text-to-image alignment, often leading to suboptimal segmentation performance. Differently, CAT-Seg [23] simultaneously fine-tunes both the text encoder and image

TABLE 1: **Comparison with state-of-the-art methods on standard benchmarks.** The best-performing results are presented in bold, while the second-best results are underlined. “E”: Euclidean Space. “H”: Hyperbolic Space.

Model	VLM	Additional Backbone	Fine-tuning Space	A-847	PC-459	A-150	PC-59	PAS-20	PAS-20 ^b
<i>Partial Fine-Tuning</i>									
ZS3Net [65]	-	ResNet-101	E	-	-	-	19.4	38.3	-
LSeg [26]	CLIP ViT-B/32	ResNet-101	E	-	-	-	-	47.4	-
OpenSeg [37]	ALIGN	ResNet-101	E	4.4	7.9	17.5	40.1	-	63.8
ZegFormer [38]	CLIP ViT-B/16	ResNet-101	E	4.9	9.1	16.9	42.8	86.2	62.7
ZSseg [39]	CLIP ViT-B/16	ResNet-101	E	7.0	-	20.5	47.7	88.4	-
OVSeg [27]	CLIP ViT-B/16	ResNet-101c	E	7.1	11.0	24.8	53.3	92.6	-
ZegCLIP [66]	CLIP ViT-B/16	-	E	-	-	-	41.2	93.6	-
<i>Selective Fine-Tuning</i>									
SED [22]	CLIP ConvNeXt-B	-	E	11.4	18.6	31.6	57.3	94.4	-
CAT-Seg [23]	CLIP ViT-B/16	-	E	12.0	19.0	31.8	57.5	94.6	77.3
<i>Parameter-efficient Fine-Tuning</i>									
SAN [24]	CLIP ViT-B/16	Side Adapter	E	10.1	12.6	27.5	53.8	94.0	-
HyperCLIP (Ours)	CLIP ViT-B/16	-	H	<u>12.3</u>	<u>19.2</u>	<u>32.1</u>	<u>58.5</u>	<u>95.6</u>	<u>78.9</u>
HyperCLIP++ (Ours)	CLIP ViT-B/16	-	H	13.1	20.3	32.9	58.6	95.8	79.6
<i>Partial Fine-Tuning</i>									
LSeg [26]	CLIP ViT-B/32	ViT-L/16	E	-	-	-	-	52.3	-
OpenSeg [37]	ALIGN	Eff-B7	E	8.1	11.5	26.4	44.8	-	70.2
OVSeg [27]	CLIP ViT-L/14	Swin-B	E	9.0	12.4	29.6	55.7	94.5	-
SAN [24]	CLIP ViT-L/14	-	E	12.4	15.7	32.1	57.7	94.6	-
ODISE [40]	CLIP ViT-L/14	Stable Diffusion	E	11.1	14.5	29.9	57.3	-	-
FC-CLIP [21]	CLIP ConvNeXt-L	-	E	14.8	18.2	34.1	58.4	95.4	-
<i>Selective Fine-Tuning</i>									
SED [22]	CLIP ConvNeXt-L	-	E	13.9	22.6	35.2	60.6	96.1	-
CAT-Seg [23]	CLIP ViT-L/14	-	E	16.0	23.8	37.9	63.3	97.0	82.5
<i>Parameter-efficient Fine-Tuning</i>									
SAN [24]	CLIP ViT-L/14	Side Adapter	E	12.4	15.7	32.1	57.7	94.6	-
HyperCLIP (Ours)	CLIP ViT-L/14	-	H	<u>16.3</u>	<u>24.1</u>	<u>38.2</u>	<u>64.2</u>	<u>98.3</u>	<u>84.3</u>
HyperCLIP++ (Ours)	CLIP ViT-L/14	-	H	17.0	25.0	38.7	64.4	98.5	84.8

TABLE 2: **Comparisons on open-vocabulary object detection task** on COCO [71]. Results are measured by the box AP at IoU threshold 0.5. “E”: Euclidean space. “H”: hyperbolic space.

Methods	Space for Fine-tuning	AP ₅₀ ^{Base}	AP ₅₀ ^{Novel}
CLIP	-	21.6	36.4
<i>Other Fine-Tuning</i>			
CLIM [72]	E	25.7	42.5
CAT-Seg [23]	E	25.2	42.3
<i>Parameter-efficient Fine-Tuning</i>			
LoRA [73]	E	24.4	41.5
HyperLoRA [54]	H	25.1	42.0
HyperCLIP [35]	H	26.3	42.7
HyperCLIP++	H	26.7	43.5

TABLE 3: **Comparisons on open-vocabulary panoptic segmentation task** on ADE20K [67]. “PQ”: Panoptic quality.

Method	FC-CLIP [21]	MAFT+ [74]	HyperCLIP	HyperCLIP++
PQ	26.8	27.1	29.2	29.9

encoder of CLIP, achieving performance comparable to ours on some of the datasets. However, its fine-tuning scheme is manually controlled through different layer combinations, necessitating a careful design to balance generalization and segmentation ability, while ours does not suffer from such an issue. Then, compared to SAN [24], another parameter-

TABLE 4: **Efficiency comparison** in terms of learnable parameters for fine-tuning CLIP. The base model is ViT-B/16.

Methods	OVSeg [27]	CAT-Seg [23]	SAN [24]	HyperCLIP++
Param. (M)	147.2	39.5	8.4	5.7

TABLE 5: Comparison on the ViT-B/32 backbone. mIoU (%) is reported on five benchmarks.

Methods	A-847	PC-459	A-150	PC-59	PAS-20 ^b
CAT-Seg [23]	9.6	15.4	26.8	50.2	80.6
HyperCLIP++	10.6	17.0	28.5	53.1	83.9

efficient fine-tuning method that introduces only a limited number of tunable parameters, our approach significantly outperforms it, achieving improvements of 6.6% on the PC-459 dataset and 4.7% on the PC-59 dataset with ViT-B/16 as the base model. These results demonstrate the effectiveness of our method in preserving generalization while mastering segmentation capability.

Application to other tasks. Here, we further carry out extra experiments in two other tasks. In an open-vocabulary object detection task, CLIP’s image embeddings are transitioned from the image level to the object level. We follow a recent work [72] to investigate open-vocabulary object detection on the COCO dataset [71]. As shown in Table 2,

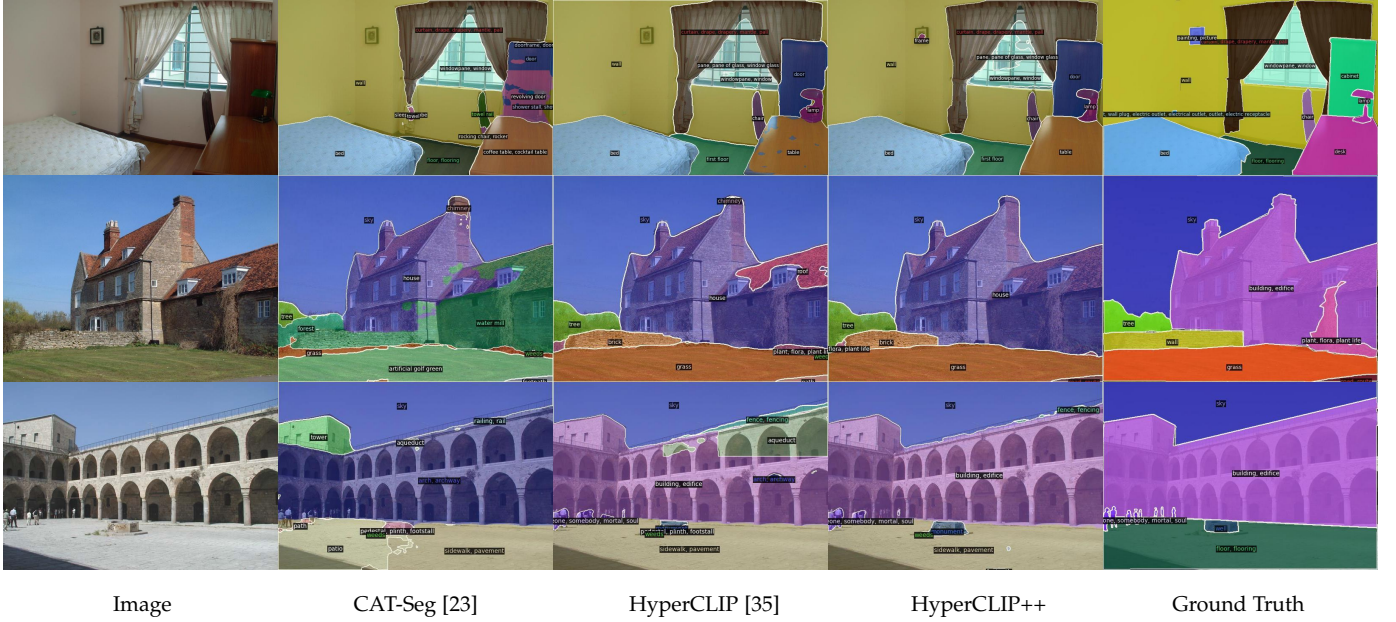


Fig. 4: Qualitative comparison of segmentation results. Compared to CAT-Seg, HyperCLIP produces notably sharper boundaries and more accurate recognition of fine-grained categories (e.g., *curtain* vs. *wall*, *shelf* vs. *cabinet*), demonstrating the benefit of hyperbolic radius adjustment for shifting both modalities toward pixel-level granularity. With the addition of DCRC, HyperCLIP++ further reduces fragmented predictions and improves spatial coherence in large regions (e.g., contiguous *house* and *grass* areas), showing that synchronized cross-modal and cross-layer communication helps resolve ambiguities that single-modality scaling alone cannot address.

the superior results demonstrate that HyperCLIP++ can effectively generalize to other tasks that require aligning text embeddings with embeddings from arbitrary hierarchical levels, e.g., object-level. Furthermore, the open-vocabulary panoptic segmentation results presented in Table 3 further corroborate this conclusion.

Relation between hyperbolic radius and granularity level.

CLIP is pre-trained with image-level supervision, whereas semantic segmentation and object detection require pixel-level and object-level representations, respectively. To investigate whether the hyperbolic radius reflects this granularity difference, we compare frozen CLIP with HyperCLIP across semantic segmentation and object detection. To isolate the contribution of radius adjustment, we report the results for HyperCLIP without DCRC. As shown in Table 6, the text/visual radii decrease from 7.5/8.2 for the original image-level representations to 6.4/7.5 after object-level adaptation and 6.1/7.2 after pixel-level adaptation, exhibiting a consistent ordering of pixel-level < object-level < image-level. Correspondingly, pixel-level adaptation improves performance on PC-459 from 6.6 to 19.2 mIoU, while object-level adaptation improves $AP_{50}^{Base}/AP_{50}^{Novel}$ on COCO from 21.6/36.4 to 26.3/42.7. These results provide direct cross-task evidence of a consistent association between the hyperbolic radius and the granularity level of CLIP’s representations.

Efficiency comparison. We compare the efficiency of our method with other approaches, including OVSeg [27], CAT-Seg [23], and SAN [24], all of which utilize a ViT version of CLIP. The comparison, summarized in Table 4, shows that our method uses the fewest trainable parameters while equipping CLIP with segmentation ability. Since we do not

TABLE 6: Relation between hyperbolic radius and granularity level. “Req. Feat. (Required/Feature)” denotes the granularity required by the task and that represented by the CLIP features, respectively. To isolate the effect of hierarchy alignment, here we report the results of HyperCLIP++ without DCRC (Ours). Semantic segmentation is evaluated on PC-459 using mIoU. Object detection is evaluated on COCO, with performance reported as $AP_{50}^{Base}/AP_{50}^{Novel}$. Perf.: performance. Text rad.: text radius. Visual rad.: visual radius.

Task	Method	Req. Feat.	Text rad.	Visual rad.	Perf.
Semantic segmentation	CLIP	Pixel/Image	7.5±0.03	8.2±0.05	6.6
	Ours	Pixel/Pixel	6.1±0.03	7.2±0.05	19.2
Object detection	CLIP	Object/Image	7.5±0.03	8.2±0.05	21.6/36.4
	Ours	Object/Object	6.4±0.03	7.5±0.05	26.3/42.7

introduce any extra architectures, and the logarithmic and exponential maps can be completed within linear complexity [54], the inference overhead is negligible.

Generalization to ViT-B/32. To further assess the architecture generalizability of our hyperbolic fine-tuning strategy, we evaluate HyperCLIP++ on the ViT-B/32 backbone. As shown in Table 5, despite the substantially reduced spatial resolution (patch size 32 vs. 16), HyperCLIP++ consistently outperforms CAT-Seg (also based on ViT-B/32) across all benchmarks, confirming that our method generalizes well to different backbone configurations.

Qualitative results. Here, we visualize our method’s representative example segmentation results against prevailing methods, i.e., CAT-Seg [23] and our conference version, i.e., HyperCLIP [35], in the A-847 dataset. As shown in Fig. 4, we can observe that our method can make more accurate

TABLE 7: **Ablation study on each component in HyperCLIP++ and comparison of universal PEFT methods**, including “LoRA” [73], “OFT” [75], “VPT” [76], “Adapter” [77], “LST” [78], “SSF” [79] and “HyperLoRA” [54] which is a recent PEFT method fine-tunes large language models in hyperbolic space. For a clear comparison, we ignore the segmentation decoder when calculating the number of learnable parameters. The base model is ViT-B/16. ✓: Fine-tuning. ✗: Freezing. DCRC: Dual cross relation communication.

Method	Image encoder	Text encoder	DCRC	Param. (M)	A-847	PC-459	A-150	PC-59	PAS-20	PAS-20 ^b
Freeze	✗	✗	✗	0	4.4	6.6	24.8	49.4	92.5	71.9
<i>Parameter-efficient Fine-Tuning</i>										
LoRA [73]	✓	✓	✗	7.5	11.4	17.6	28.6	55.1	94.2	76.7
OFT [75]	✓	✓	✗	3.8	10.9	18.0	30.2	53.7	93.7	74.3
VPT [76]	✓	✓	✗	2.2	5.7	10.2	23.7	54.3	93.8	75.1
Adapter [77]	✓	✓	✗	7.5	10.4	16.5	28.8	54.9	94.2	75.2
SSF [79]	✓	✓	✗	0.6	6.9	15.2	28.6	52.1	93.2	72.8
HyperLoRA [54]	✓	✓	✗	7.5	11.4	18.2	29.7	55.8	94.3	77.2
HyperCLIP++ (Ours)	✓	✗	✗	3.4	10.3	16.8	30.5	57.6	94.4	78.0
	✗	✗	✗	2.2	11.1	17.5	29.8	55.1	93.7	76.1
	✓	✓	✗	5.6	12.3	19.2	32.1	58.5	95.6	78.9
	✓	✓	✓	5.7	13.1	20.3	32.9	58.6	95.8	79.6

TABLE 8: **Ablation on the size of diagonal blocks n** . The base model is ViT-B/16. The best-performing results are presented in bold, the second-best results are underlined.

n	Param. (M)	A-847	PC-459	A-150	PC-59	PAS-20 ^b
1	0.04	6.0	9.5	27.4	52.4	75.6
8	0.36	10.2	15.8	30.7	54.4	76.2
16	0.72	11.6	17.9	31.3	56.2	76.9
32	1.45	12.0	18.8	32.1	57.0	77.4
64	2.87	12.6	19.6	32.3	58.3	78.6
128	5.69	13.1	20.3	32.9	<u>58.6</u>	79.6
256	11.38	<u>13.0</u>	<u>20.1</u>	<u>32.8</u>	58.8	<u>79.6</u>

and smooth predictions on object location and category. Even if objects are small and the background is complicated, our method still performs well.

6.3 Ablation Study

Ablation of fine-tuning CLIP. We assess the efficacy of various fine-tuning design choices for HyperCLIP. As shown in Table 7, we freeze one of CLIP’s encoders and compare the performance with several commonly-used PEFT methods, e.g., LoRA [73]. Compared to fully freezing CLIP’s encoders (row 1), fine-tuning either encoder independently allows for minimizing its hyperbolic radius, thereby equipping the corresponding encoder with segmentation ability. However, one modality’s hierarchical level remains at the image level, which hinders CLIP from fully acquiring segmentation ability. Consequently, jointly fine-tuning both encoders aligns the hierarchical levels at the pixel level, resulting in better performance. Finally, the incorporation of the Dual Cross-Relation Communication (DCRC) module enforces cross-modal hierarchy alignment during training, yielding optimal performance. We also compare our method with existing PEFT strategies. Most of these methods, which operate in Euclidean space, struggle to capture the transition between different hierarchical levels. Notably, Hyper-

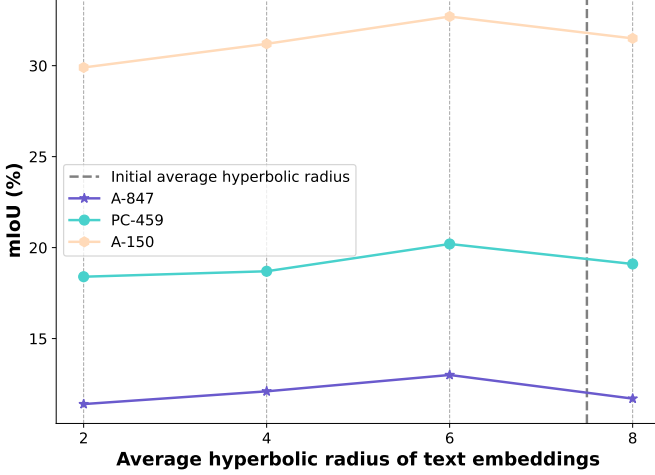
LoRA [54] also updates parameters in hyperbolic space. However, because HyperLoRA does not specifically adjust the hyperbolic radius, it fails to fully endow CLIP with segmentation ability, resulting in inferior performance compared to our method.

Comparison with other geometric learning approaches. Our hyperbolic radius adjustment provides a unique advantage over fine-tuning in alternative geometric spaces. In Euclidean space, standard PEFT methods (e.g., LoRA, Adapter) modify embeddings without any geometric constraint on hierarchical structure, treating the hyperbolic radius as a free variable with no semantic interpretation. In hyperspherical space, orthogonal fine-tuning methods (e.g., OFT) maintain angular relationships but fundamentally prevents radius adjustment—the very mechanism that enables hierarchy alignment in our framework. In hyperbolic space, our block-diagonal scaling matrices offer direct, theoretically grounded control over the Poincaré ball radius (Theorem 1), allowing us to explicitly bridge the gap between CLIP’s original image-level granularity and the pixel-level granularity required for segmentation. This controllability distinguishes our approach from prior hyperbolic methods such as HyperLoRA, which perform low-rank updates in hyperbolic space but do not explicitly adjust the radius.

Parameter Efficiency vs. Accuracy Trade-off. Table 8 reveals a clear trade-off between parameter efficiency and segmentation accuracy as the block size n varies from 1 to 256. On benchmarks with moderate category counts (PC-59, PAS-20^b), accuracy improves monotonically with increasing block size, since larger blocks provide richer cross-dimensional interactions for radius adjustment. However, on the most challenging benchmark A-847 (847 classes), performance peaks at $n = 128$ (5.69M parameters) and slightly degrades at $n = 256$ (11.38M). This saturation indicates that excessive learnable parameters might risk overfitting to the 171 training classes, thereby harming open-vocabulary generalization to unseen classes. Based on these findings, we recommend $n = 128$ as the default configuration, which achieves the best overall balance with only 5.7M trainable

TABLE 9: **Ablation on the necessity of Möbius matrix multiplication.** “Möbius.”: Möbius matrix multiplication.

Möbius.	A-847	PC-459	A-150	PC-59	PAS-20 ^b
✗	12.0	18.9	31.2	57.5	78.2
✓	13.1	20.3	32.9	58.6	79.6

Fig. 5: **Visualization on the effect of different average hyperbolic radii of text embeddings.** The controlling of different hyperbolic radii is achieved via a mean squared error (MSE) loss that enforces a constraint between the expected radius and the mean radius of the text embeddings during fine-tuning.

parameters—roughly 4% of the full CLIP ViT-B/16 model (149M). For deployment scenarios with stricter parameter budgets, $n = 64$ (1.42M) provides a competitive alternative with only marginal accuracy loss.

Effect of the hyperbolic radius of text embeddings. In order to closely examine the effect of the hyperbolic radius on equipping CLIP with segmentation ability, we apply a mean squared error (MSE) loss during fine-tuning. This MSE loss encourages the average hyperbolic radius of the text embeddings to converge towards different target values. As shown in Fig. 5, the initial value of the average hyperbolic radius is approximately 7.5, which is subsequently minimized to around 6.0 to achieve the optimal value for segmentation. Deviating from this value, either by increasing or decreasing the expected radius, results in diminished performance. This observation offers a new insight: the hyperbolic radius required for a vision task might be relatively fixed, suggesting that it could serve as a metric to quantify the granularity level of different vision tasks.

Necessity of Möbius matrix multiplication. We ablate this experiment to validate the influence of the Möbius matrix multiplication operation. In Table 9, “✗” means that we use standard matrix multiplication. It achieves worse performance compared with “✓”, which indicates that the Möbius matrix multiplication is essential for minimizing the hyperbolic radius of CLIP’s embeddings.

Ablation of dual cross-relation communications. Table 10 presents the ablation results of three cross-relation communication methods. The dual cross-relation communication

TABLE 10: **Ablation on three ways of cross-relation communications.** DCRC: Dual cross-relation communications, which contains both cross-modality and cross-layer.

Communication	A-847	PC-459	A-150	PC-59	PAS-20 ^b
Cross-modality	12.9	20.1	32.6	58.5	95.7
Cross-layer	12.5	19.5	32.3	58.6	95.8
DCRC (Ours)	13.1	20.3	32.9	58.6	95.8

TABLE 11: Comparison of cross-modal communication mechanisms. The base model is ViT-B/16.

Communication	Par. (M)	A-847	PC-459	A-150	PC-59	PAS-20 ^b
Late fusion (w/o DCRC)	0	12.3	19.2	32.1	58.5	78.9
Cross-attn adapter [80]	0.4	12.7	19.6	32.5	58.5	79.1
DCRC (Ours)	0.1	13.1	20.3	32.9	58.6	79.6

(DCRC) achieves the best performance on most datasets, particularly reaching accuracies of 13.1%, 20.3%, and 32.9% on the A-847, PC-459, and A-150 datasets, respectively, significantly outperforming standalone cross-modality or cross-layer communication. This demonstrates that simultaneously considering inter-modal and inter-layer information interaction can more effectively enhance model performance. Notably, on the PC-59 and PAS-20^b datasets, the three methods exhibit comparable performance, suggesting that for certain specific tasks, a single communication approach may suffice.

Comparison with cross-modal communication baselines. To validate the design of our DCRC module, we compare against two cross-modal communication strategies in Table 11: (1) *Late fusion* (i.e., HyperCLIP++ w/o DCRC), where the vision and text scaling matrices are independently optimized and only interact at the final classification layer; and (2) *Cross-attention adapter* [80], which inserts bidirectional cross-attention modules between the vision and text pathways at each encoder stage, enabling each modality to attend to the other’s intermediate features. Despite using 4× more parameters (0.4M vs. 0.1M), the cross-attention adapter only achieves marginal improvement over late fusion. This is because it operates on a per-layer basis and lacks the global cross-layer communication that our DCRC provides through the higher-order T-product. In contrast, DCRC jointly models cross-modality and cross-layer interactions in a single structured tensor operation, achieving the best performance with the fewest additional parameters.

Layers of MLP in DCRC. Table [?] investigates the impact of the number of MLP layers k in the DCRC module. When $k = 2$, the model performs optimally on the A-847, A-150, and PAS-20^b datasets, while $k=3$ shows slight advantages on PC-459 and PC-59. This indicates that tasks of varying complexity require different depths of feature interaction: simpler tasks (e.g., PAS-20^b) may benefit from shallow interactions, whereas more complex tasks (e.g., PC-459) demand deeper information fusion. Considering both computational cost and performance, we ultimately select $k = 2$ as the default configuration.

TABLE 12: Ablation on the impact of different numbers of layers k of MLP in DCRC module.

layers	A-847	PC-459	A-150	PC-59	PAS-20 ^b
$k = 1$	12.8	20.1	32.6	58.5	95.7
$k = 2$	13.1	20.3	32.9	58.6	95.8
$k = 3$	12.7	20.5	32.4	58.7	95.7

TABLE 13: Training and inference overheads of different model variants on the ViT-B/16 backbone. Scaling T.: scaling transformations. Tr. Time: Training Time.

Metrics	CAT-Seg [23]	HyperCLIP++ w/o DCRC w/o Scaling T.	HyperCLIP++ w/o DCRC	HyperCLIP++
Params (M) ↓	39.5	0	5.6	5.7
Tr. Time (h) ↓	8.5	6	7.5	8
FLOPs (G) ↓	1837.2	1837.2	1895.9	1953.6
Latency (s) ↓	0.163	0.163	0.175	0.179

Training and Inference Overheads. Table 13 provides a detailed breakdown of computational costs across model variants on the ViT-B/16 backbone. CAT-Seg directly fine-tunes some selected CLIP layers, resulting in 39.5M trainable parameters and a training time of 8.5 hours, yet its inference FLOPs and latency remain identical to the frozen baseline (1837.2G / 0.163s) since no extra modules are introduced at test time. In contrast, HyperCLIP++ introduces only 5.7M learnable parameters (5.6M for the block-diagonal scaling matrices plus 0.1M for DCRC). The scaling transformations add 58.7G FLOPs (3.2%) and 0.012s latency due to the exponential/logarithmic maps and Möbius matrix multiplication; The DCRC module contributes an additional 57.7G FLOPs (3.0%) and only 0.004s latency. Overall, HyperCLIP++ achieves a favorable trade-off: it uses $\sim 7\times$ fewer trainable parameters than CAT-Seg with comparable training time, at the cost of a modest 6.3% increase in inference FLOPs and a 0.016s increase in latency.

Visualization of Hyperbolic Radius. To provide an intuitive validation of the Dual Cross-Relation Communication (DCRC) module’s effectiveness in synchronizing modalities, we visualize the hyperbolic radius distributions of CLIP’s embeddings after fine-tuning. As shown in Figure 6, we compare our full HyperCLIP++ model against its variant without the DCRC module (denoted as HyperCLIP). It is evident that without DCRC, the independent adaptation of each pathway leads to a noticeable discrepancy between the radius distributions for text (Fig. 6 (a)) and visual (Fig. 6 (b)) embeddings. By incorporating the DCRC module, HyperCLIP++ significantly shifts both distributions, resulting in a much closer alignment of hyperbolic radii between text and visual embeddings. This enhanced alignment in hyperbolic radii directly translates to improved hierarchical alignment of the modalities. This suggests that the DCRC module successfully enforces a more consistent and coherent embedding geometry across the vision and text pathways, validating its crucial role in maintaining cross-modal alignment necessary for optimal final performance.

Robustness under domain shifts. We further evaluate cross-domain generalization on the ISPRS Potsdam [81]

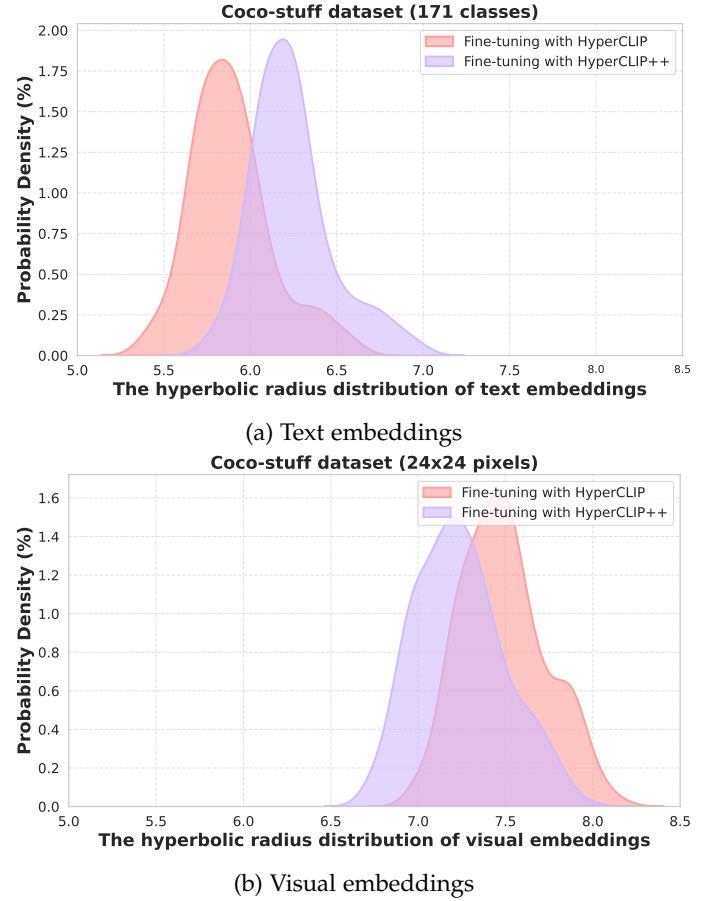


Fig. 6: Visualization of the hyperbolic radius distributions of CLIP embeddings after fine-tuning. In both (a) text and (b) visual embeddings, HyperCLIP++ shifts the distributions compared with HyperCLIP. Notably, the two modalities exhibit **closer alignment** in their hyperbolic radii under HyperCLIP++, suggesting an improved modality alignment.

TABLE 14: Cross-domain evaluation on two representative remote-sensing datasets. All models are trained exclusively on COCO-Stuff and evaluated without target-domain fine-tuning or annotations.

Method	Potsdam [81]	Vaihingen [82]
CAT-Seg [23]	42.7	34.9
HyperCLIP++	44.8	36.5

and Vaihingen [82] remote-sensing datasets. All models are trained exclusively on COCO-Stuff and directly evaluated on the target datasets without target-domain fine-tuning or annotations. As shown in Table 14, HyperCLIP++ achieves 44.8 and 36.5 mIoU on Potsdam and Vaihingen, outperforming CAT-Seg by 2.1 and 1.6 mIoU points, respectively. These results demonstrate the robustness of HyperCLIP++ beyond natural-image benchmarks under substantial domain shifts.

Limitations. While HyperCLIP++ effectively adapts CLIP at the pixel level, it shares a fundamental limitation with all parameter-efficient fine-tuning methods: the adaptation process inevitably sacrifices some of CLIP’s original zero-shot generalization. This manifests primarily as inter-class

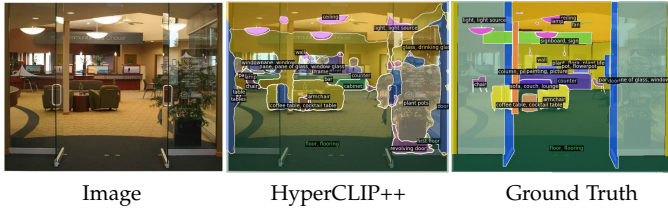


Fig. 7: Failure case of HyperCLIP++ on a complex indoor scene from ADE20K. The model exhibits inter-class confusion among visually similar categories (e.g., furniture and structural elements) under cluttered backgrounds, a limitation shared by all fine-tuning methods.

confusion in complex scenes where many visually similar categories co-occur under cluttered backgrounds. As shown in Fig. 7, the model occasionally misclassifies similar categories in dense indoor environments. Notably, HyperCLIP++ mitigates this trade-off better than alternatives by introducing the fewest trainable parameters (5.7M), thereby preserving the vast majority of CLIP’s pre-trained representations. Future work may explore dynamic, context-aware radius hyperbolic radius adjustment or hierarchical class grouping strategies to further alleviate inter-class ambiguity in complex scenes.

7 CONCLUSION

In this paper, we introduced HyperCLIP++, a novel and parameter-efficient fine-tuning strategy that addresses the mismatch between CLIP’s pre-trained image-level granularity and the pixel-level granularity required by open-vocabulary semantic segmentation. We observed that aligning the hierarchical level of CLIP’s text encoder with targeted tasks, i.e., segmentation, is crucial for accurate pixel-level predictions. HyperCLIP++ achieves this by (1) adjusting the hierarchical level in hyperbolic space through scaling transformations applied to CLIP’s embeddings, enabling precise cross-modal alignment, and (2) employing a novel Dual Cross-Relation Communication (DCRC) module to maintain cross-modal alignment during hierarchy alignment. Extensive experiments demonstrate that HyperCLIP++ sets new state-of-the-art results in open-vocabulary semantic segmentation. More importantly, we observe that after adjustment, CLIP’s text embeddings maintain a relatively fixed hyperbolic radius across datasets. This suggests that the segmentation task’s granularity level may be quantified using the hyperbolic radius, offering a new direction for multi-modal learning.

REFERENCES

- [1] H. Tan and M. Bansal, “Lxmert: Learning cross-modality encoder representations from transformers,” *arXiv:1908.07490*, 2019.
- [2] Y.-C. Chen, L. Li, L. Yu *et al.*, “Uniter: Universal image-text representation learning,” in *ECCV*, 2020.
- [3] J. Lu, D. Batra, D. Parikh *et al.*, “Vilbert: Pretraining task-agnostic visiolinguistic representations for vision-and-language tasks,” in *NeurIPS*, 2019.
- [4] A. Radford, J. W. Kim, C. Hallacy *et al.*, “Learning transferable visual models from natural language supervision,” in *ICML*, 2021.
- [5] Z.-Y. Dou, Y. Xu, Z. Gan *et al.*, “An empirical study of training end-to-end vision-and-language transformers,” in *CVPR*, 2022.
- [6] Z. Huang, Z. Zeng, Y. Huang *et al.*, “Seeing out of the box: End-to-end pre-training for vision-language representation learning,” in *CVPR*, 2021.
- [7] J. Li, D. Li, C. Xiong *et al.*, “Blip: Bootstrapping language-image pre-training for unified vision-language understanding and generation,” in *ICML*, 2022.
- [8] W. Kim, B. Son, and I. Kim, “Vilt: Vision-and-language transformer without convolution or region supervision,” in *ICML*, 2021.
- [9] J. Li, R. Selvaraju, A. Gotmare *et al.*, “Align before fuse: Vision and language representation learning with momentum distillation,” in *NeurIPS*, 2021.
- [10] Z. Gan, Y.-C. Chen, L. Li *et al.*, “Large-scale adversarial training for vision-and-language representation learning,” in *NeurIPS*, 2020.
- [11] A. Kamath, M. Singh, Y. LeCun *et al.*, “Mdettr-modulated detection for end-to-end multi-modal understanding,” in *ICCV*, 2021.
- [12] X. Li, X. Yin, C. Li *et al.*, “Oscar: Object-semantics aligned pre-training for vision-language tasks,” in *ECCV*, 2020.
- [13] Z.-Y. Dou, A. Kamath, Z. Gan *et al.*, “Coarse-to-fine vision-language pre-training with fusion in the backbone,” in *NeurIPS*, 2022.
- [14] L. H. Li, P. Zhang, H. Zhang *et al.*, “Grounded language-image pre-training,” in *CVPR*, 2022.
- [15] J. Wang, X. Hu, Z. Gan *et al.*, “Ufo: A unified transformer for vision-language representation learning,” *arXiv:2111.10023*, 2021.
- [16] Z. Peng, Z. Xu, Z. Zeng *et al.*, “Parameter efficient fine-tuning via cross block orchestration for segment anything model,” in *CVPR*, 2024.
- [17] —, “Sam-parser: Fine-tuning sam efficiently by parameter space reconstruction,” in *AAAI*, 2024.
- [18] M. Hu, K. Yuan, Y. Shen *et al.*, “Ophclip: Hierarchical retrieval-augmented learning for ophthalmic surgical video-language pre-training,” *arXiv:2411.15421*, 2024.
- [19] F. Tang, Z. Huang, C. Liu *et al.*, “Intervening anchor token: Decoding strategy in alleviating hallucinations for mllms,” in *ICLR*, 2025.
- [20] H. Xue, F. Tang, M. Hu *et al.*, “Mmr: A large-scale benchmark for understanding multimodal large language model in real-world conversation,” *arXiv:2502.11903*, 2025.
- [21] Q. Yu, J. He, X. Deng *et al.*, “Convolutions die hard: Open-vocabulary segmentation with single frozen convolutional clip,” in *NeurIPS*, 2023.
- [22] B. Xie, J. Cao, J. Xie *et al.*, “Sed: A simple encoder-decoder for open-vocabulary semantic segmentation,” in *CVPR*, 2024.
- [23] S. Cho, H. Shin, S. Hong *et al.*, “Cat-seg: Cost aggregation for open-vocabulary semantic segmentation,” in *CVPR*, 2024.
- [24] M. Xu, Z. Zhang, F. Wei *et al.*, “Side adapter network for open-vocabulary semantic segmentation,” in *CVPR*, 2023.
- [25] H. Caesar, J. Uijlings, and V. Ferrari, “Coco-stuff: Thing and stuff classes in context,” in *CVPR*, 2018.
- [26] B. Li, K. Q. Weinberger, S. Belongie *et al.*, “Language-driven semantic segmentation,” in *ICLR*, 2022.
- [27] F. Liang, B. Wu, X. Dai *et al.*, “Open-vocabulary semantic segmentation with mask-adapted clip,” in *CVPR*, 2023.
- [28] Z. Peng, Z. Xu, Z. Zeng *et al.*, “Parameter-efficient fine-tuning in hyperspherical space for open-vocabulary semantic segmentation,” *arXiv:2405.18840*, 2024.
- [29] J. W. Cannon, W. J. Floyd, R. Kenyon *et al.*, “Hyperbolic geometry,” *Flavors of geometry*, 1997.
- [30] I. Biederman, “Recognition-by-components: a theory of human image understanding,” *Psychological review*, 1987.
- [31] M. Weber, M. Zaheer, A. S. Rawat *et al.*, “Robust large-margin learning in hyperbolic space,” in *NeurIPS*, 2020.
- [32] F. Sala, C. De Sa, A. Gu *et al.*, “Representation tradeoffs for hyperbolic embeddings,” in *ICML*, 2018.
- [33] S. Ramasinghe, V. Shevchenko, G. Avraham *et al.*, “Accept the modality gap: An exploration in the hyperbolic space,” in *CVPR*, 2024.
- [34] D. Surís, R. Liu, and C. Vondrick, “Learning the predictability of the future,” in *CVPR*, 2021.
- [35] Z. Peng, Z. Xu, Z. Zeng *et al.*, “Understanding fine-tuning clip for open-vocabulary semantic segmentation in hyperbolic space,” in *CVPR*, 2025.
- [36] C. Zhou, C. C. Loy, and B. Dai, “Extract free dense labels from clip,” in *ECCV*, 2022.
- [37] G. Ghiasi, X. Gu, Y. Cui *et al.*, “Scaling open-vocabulary image segmentation with image-level labels,” in *ECCV*, 2022.

- [38] J. Ding, N. Xue, G.-S. Xia *et al.*, “Decoupling zero-shot semantic segmentation,” in *CVPR*, 2022.
- [39] M. Xu, Z. Zhang, F. Wei *et al.*, “A simple baseline for open-vocabulary semantic segmentation with pre-trained vision-language model,” in *ECCV*, 2022.
- [40] J. Xu, S. Liu, A. Vahdat *et al.*, “Open-vocabulary panoptic segmentation with text-to-image diffusion models,” in *CVPR*, 2023.
- [41] B. Cheng, I. Misra, A. G. Schwing *et al.*, “Masked-attention mask transformer for universal image segmentation,” in *CVPR*, 2022.
- [42] S. Jiao, H. Zhu, J. Huang *et al.*, “Collaborative vision-text representation optimizing for open-vocabulary segmentation,” in *ECCV*, 2024.
- [43] X. Shan, D. Wu, G. Zhu *et al.*, “Open-vocabulary semantic segmentation with image embedding balancing,” in *CVPR*, 2024.
- [44] M. R. Bridson and A. Haeffliger, *Metric spaces of non-positive curvature*. Springer, 2013.
- [45] Z. Yu, T. Nguyen, Y. Gal *et al.*, “Skin lesion recognition with class-hierarchy regularized hyperbolic embeddings,” in *MICCAI*, 2022.
- [46] J. Chen, J. Qin, Y. Shen *et al.*, “Learning attentive and hierarchical representations for 3d shape recognition,” in *ECCV*, 2020.
- [47] S. Wang, Q. Kang, R. She *et al.*, “Hypilloc: Towards effective lidar pose regression with hyperbolic fusion,” in *CVPR*, 2023.
- [48] O. Ganea, G. Bécigneul, and T. Hofmann, “Hyperbolic neural networks,” in *NeurIPS*, 2018.
- [49] Y. Zhu, D. Zhou, J. Xiao *et al.*, “Hypertext: Endowing fasttext with hyperbolic geometry,” *arXiv:2010.16143*, 2020.
- [50] V. Khrulkov, L. Mirvakhabova, E. Ustinova *et al.*, “Hyperbolic image embeddings,” in *CVPR*, 2020.
- [51] Y. Guo, X. Wang, Y. Chen *et al.*, “Clipped hyperbolic classifiers are super-hyperbolic classifiers,” in *CVPR*, 2022.
- [52] K. Desai, M. Nickel, T. Rajpurohit *et al.*, “Hyperbolic image-text representations,” in *ICML*, 2023.
- [53] F. Kong, Y. Chen, J. Cai *et al.*, “Hyperbolic learning with synthetic captions for open-world detection,” in *CVPR*, 2024.
- [54] M. Yang, A. Feng, B. Xiong *et al.*, “Hyperbolic fine-tuning for large language models,” *arXiv:2410.04010*, 2024.
- [55] C. Hu, K.-Y. Zhang, T. Yao *et al.*, “Rethinking generalizable face anti-spoofing via hierarchical prototype-guided distribution refinement in hyperbolic space,” in *CVPR*, 2024.
- [56] M. Nickel and D. Kiela, “Learning continuous hierarchies in the lorentz model of hyperbolic geometry,” in *ICML*, 2018.
- [57] A. A. Ungar, “Hyperbolic trigonometry and its application in the poincaré ball model of hyperbolic geometry,” *Comput. Math. Appl.*, 2001.
- [58] —, “A gyrovector space approach to hyperbolic geometry,” *Synthesis Lectures on Mathematics and Statistics*, 2008.
- [59] J. Lee, J. Choi, J. Mok *et al.*, “Reducing information bottleneck for weakly supervised semantic segmentation,” in *NeurIPS*, 2021.
- [60] Z. Wang, X. Chen, R. Wen *et al.*, “Information theoretic counterfactual learning from missing-not-at-random feedback,” in *NeurIPS*, 2020.
- [61] Z. Peng, Z. Xu, Z. Zeng *et al.*, “Parameter efficient fine-tuning via cross block orchestration for segment anything model,” in *CVPR*, 2024.
- [62] E. Kernfeld, M. Kilmer, and S. Aeron, “Tensor–tensor products with invertible linear transforms,” *Linear Algebra Appl.*, 2015.
- [63] M. E. Kilmer and C. D. Martin, “Factorization strategies for third-order tensors,” *Linear Algebra Appl.*, 2011.
- [64] C. Lu, X. Peng, and Y. Wei, “Low-rank tensor completion with a new tensor nuclear norm induced by invertible linear transforms,” in *CVPR*, 2019.
- [65] M. Bucher, T.-H. Vu, M. Cord *et al.*, “Zero-shot semantic segmentation,” in *NeurIPS*, 2019.
- [66] Z. Zhou, Y. Lei, B. Zhang *et al.*, “Zegclip: Towards adapting clip for zero-shot semantic segmentation,” in *CVPR*, 2023.
- [67] B. Zhou, H. Zhao, X. Puig *et al.*, “Semantic understanding of scenes through the ade20k dataset,” *IJCV*, 2019.
- [68] M. Everingham, L. Van Gool, C. K. Williams *et al.*, “The pascal visual object classes (voc) challenge,” *IJCV*, 2010.
- [69] R. Mottaghi, X. Chen, X. Liu *et al.*, “The role of context for object detection and semantic segmentation in the wild,” in *CVPR*, 2014.
- [70] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *arXiv:1412.6980*, 2014.
- [71] T.-Y. Lin, M. Maire, S. Belongie *et al.*, “Microsoft coco: Common objects in context,” in *ECCV*, 2014.
- [72] S. Wu, W. Zhang, L. Xu *et al.*, “Clim: Contrastive language-image mosaic for region representation,” in *AAAI*, 2024.
- [73] E. J. Hu, Y. Shen, P. Wallis *et al.*, “LoRA: Low-rank adaptation of large language models,” in *ICLR*, 2022.
- [74] S. Jiao, H. Zhu, J. Huang *et al.*, “Collaborative vision-text representation optimizing for open-vocabulary segmentation,” in *ECCV*, 2024.
- [75] Z. Qiu, W. Liu, H. Feng *et al.*, “Controlling text-to-image diffusion by orthogonal finetuning,” in *NeurIPS*, 2023.
- [76] M. Jia, L. Tang, B.-C. Chen *et al.*, “Visual prompt tuning,” in *ECCV*, 2022.
- [77] N. Houlsby, A. Giurgiu, S. Jastrzebski *et al.*, “Parameter-efficient transfer learning for nlp,” in *ICML*, 2019.
- [78] Y.-L. Sung, J. Cho, and M. Bansal, “Lst: Ladder side-tuning for parameter and memory efficient transfer learning,” in *NeurIPS*, 2022.
- [79] D. Lian, D. Zhou, J. Feng *et al.*, “Scaling & shifting your features: A new baseline for efficient model tuning,” in *NeurIPS*, 2022.
- [80] Z. Xu, Z. Chen, Y. Zhang *et al.*, “Bridging vision and language encoders: Parameter-efficient tuning for referring image segmentation,” in *ICCV*, 2023.
- [81] International Society for Photogrammetry and Remote Sensing, “2D Semantic Labeling Contest—Potsdam,” <https://www.isprs.org/resources/datasets/benchmarks/UrbanSemLab/2d-sem-label-potsdam.aspx>.
- [82] —, “2D Semantic Labeling—Vaihingen Data,” <https://www.isprs.org/resources/datasets/benchmarks/UrbanSemLab/2d-sem-label-vaihingen.aspx>.